

Gestione dei Dati dei Pazienti con Snowflake

Progetto di Data Engineering con Snowflake

Autore: Federico Vita

Data: 14/06/2026

Indice dei Contenuti

1	Contesto e obiettivo	4
1.1	Contesto	4
1.2	Obiettivo	4
2	Dati sintetici	4
2.1	Motivazione e approccio	4
2.2	Sorgente: healthdata-synthetic-generator e SDV	4
2.3	Organizzazione per domini	5
2.4	Dizionario dati	5
2.4.1	Dominio ERP	5
2.4.2	Dominio IoT	5
2.4.3	Dominio EHR	6
2.5	Relazioni e integrità referenziale	7
2.6	Ruoli, permessi e predisposizione per integrazione Snowflake	7
2.6.1	Bucket S3 e struttura interna dei dati	7
2.6.2	Ruolo IAM dedicato a Snowflake	8
2.6.3	Policy IAM (Least Privilege)	8
2.6.4	Trust policy del ruolo IAM (predisposta con placeholder)	9
3	Architettura logica su Snowflake	9
3.1	Visione d'insieme	9
3.2	Creazione del database principale	9
3.3	Implementazione del Data Layering (Schemi)	10
3.4	Configurazione delle risorse di calcolo (Virtual Warehouse)	10
3.4.1	Warehouse per Ingestion	10
3.4.2	Warehouse per Operazioni (Consultazione)	10
3.4.3	Warehouse per Analisi (Analytics)	11
3.5	Riepilogo assetto architetturale	11
4	Progettazione del modello dati	11
4.1	Obiettivo del modello e criteri di progettazione	11
4.2	Layer RAW – Tabelle di atterraggio (Mirror)	11
4.2.1	RAW – Dominio ERP (Enterprise Resource Planning)	11
4.2.2	RAW – Dominio IoT (Internet of Things)	12
4.2.3	RAW – Dominio EHR (Electronic Health Record)	13
4.3	Layer CURATED – Standardizzazione e qualità	13
4.3.1	CURATED – Tabelle Anagrafiche e Strutturali (ERP Key)	14
4.3.2	CURATED – Dispositivi e Parametri Vitali (IoT)	14
4.3.3	CURATED – Pazienti e Ricoveri (EHR)	14
4.3.4	CURATED – Gestione Anomalie (Quarantena)	15
4.4	Layer ANALYTICS – Modello dimensionale	15
4.4.1	ANALYTICS – Dimensioni	15
4.4.2	ANALYTICS – Tabelle dei Fatti e Bridge	16
4.5	Conclusioni	17
5	Sicurezza e conformità normativa	18
5.1	7.1 Principi di sicurezza applicati al modello	18
5.2	7.2 Strategia RBAC su Snowflake	19
5.2.1	Gerarchia dei ruoli	19
5.2.2	Assegnazione ruoli all'utente di lab	19
5.3	7.3 Assegnazione privilegi per layer	20
5.3.1	Privilegi sui Virtual Warehouse (coerenza con Sezione 5)	20

5.3.2	CURATED	20
5.3.3	ANALYTICS	21
5.3.4	PIPELINE	21
5.4	7.4 Protezione dei dati sensibili (PII)	22
5.5	7.5 Principio del minimo privilegio e governance	23
6	Caricamento dati e pipeline	23
6.1	Obiettivo	23
6.2	Ingestione e staging	23
6.3	Pipeline ELT (flusso unico: task → procedure)	25
6.3.1	1) RAW: sp_load_raw() (COPY INTO, fail-fast)	25
6.3.2	2) CURATED: sp_transform_curated() (quarantena + MERGE, idempotente) . . .	26
6.3.3	3) ANALYTICS: sp_publish_analytics() (solo MERGE, idempotente)	34
6.4	Orchestrazione e scheduling (HEALTHCARE_DW.PIPELINE)	36
6.4.1	Grant strutturali (staging + schema)	36
6.4.2	Grant runtime (warehouse + execute + operate)	37
6.4.3	Ownership e delega operativa (EXECUTE AS OWNER)	38
7	Validazione analitica e query dimostrative	38
7.1	Obiettivo della validazione	38
7.2	1. Verifica tecnica del flusso (Data Engineering)	39
7.3	2. Verifica gestione anomalie (Quarantena)	39
7.4	3. Analisi di Business (Data Analyst)	40
7.5	4. Verifica Masking Policy (Security & Compliance)	40
8	Conclusioni	41
8.1	Sintesi del lavoro svolto	41
8.2	Benefici della soluzione	41
8.3	Considerazioni finali	41

1 Contesto e obiettivo

1.1 Contesto

HealthDataPro è un'azienda leader nella fornitura di soluzioni di gestione e analisi dati per il settore sanitario. L'obiettivo è supportare le strutture ospedaliere nell'ottimizzare la gestione dei dati clinici, migliorando la qualità dell'assistenza e la sicurezza dei pazienti.

Un grande ospedale situato in un'area urbana ha espresso la necessità di modernizzare il proprio sistema di gestione dei dati dei pazienti. Attualmente, i dati clinici sono distribuiti tra diversi sistemi legacy, rendendo difficoltosi:

- il consolidamento delle informazioni;
- l'analisi;
- l'accesso in tempo reale alle informazioni critiche.

1.2 Obiettivo

Creare un sistema centralizzato basato su **Snowflake** per gestire in modo **sicuro** e **scalabile** i dati dei pazienti. L'obiettivo è consentire al personale medico e amministrativo di accedere rapidamente alle informazioni, garantendo al contempo il massimo livello di **privacy** e la conformità normativa (**GDPR**).

Nota: l'architettura e il modello dati dovranno supportare controlli di accesso granulari e protezione dei dati sensibili, in linea con i requisiti di sicurezza e conformità.

2 Dati sintetici

2.1 Motivazione e approccio

Nel contesto di questo progetto, l'utilizzo di dati sanitari reali comporterebbe significative complessità legali ed etiche, in particolare legate al rispetto della privacy dei pazienti e alla conformità con normative rigorose come il GDPR. Per ovviare a queste problematiche e garantire la massima flessibilità nella fase di sviluppo e test, ho optato per la generazione di un dataset interamente sintetico.

Questa scelta mi ha permesso di disporre di una base dati verosimile e strutturata, necessaria per convalidare l'architettura Snowflake e le pipeline di ingestione, eliminando alla radice il rischio di esporre informazioni sensibili. La fase di generazione dei dati costituisce quindi il punto di partenza fondamentale dell'intero progetto, definendo il perimetro informativo su cui si baseranno tutte le analisi successive.

2.2 Sorgente: healthdata-synthetic-generator e SDV

Per produrre il dataset, ho utilizzato un progetto esterno denominato «healthdata-synthetic-generator», basato sulla libreria Python **SDV (Synthetic Data Vault)**. SDV è uno strumento avanzato che consente di modellare dataset multi-tabella, apprendendo le distribuzioni statistiche e le relazioni dai dati originali (o da schemi definiti) per generare nuovi record che mantengono la coerenza referenziale e le proprietà statistiche.

Il generatore è stato configurato per rispettare regole di business specifiche e vincoli di integrità. Il codice sorgente e la documentazione tecnica del generatore sono disponibili al seguente link: [healthdata-synthetic-generator](#)

2.3 Organizzazione per domini

Ho strutturato il dataset organizzandolo in tre domini logici distinti, per simulare la natura eterogenea dei sistemi informativi ospedalieri reali. Questa suddivisione riflette la necessità di consolidare fonti diverse all'interno del Data Warehouse:

- **EHR (Electronic Health Record)**: Contiene i dati clinici fondamentali, inclusi i dettagli sui pazienti, i ricoveri ospedalieri e le diagnosi associate.
- **ERP (Enterprise Resource Planning)**: Raccoglie i dati operativi e organizzativi, come la gestione dei reparti, l'anagrafica del personale e i turni di assegnazione.
- **IoT (Internet of Things)**: Include i dati generati dai dispositivi medici e le relative misurazioni dei parametri vitali.

2.4 Dizionario dati

Di seguito riporto il dizionario dati completo, suddiviso per dominio, che descrive le entità generate e il significato di ciascun campo.

2.4.1 Dominio ERP

1. **Reparti («reparti»)** Rappresenta le unità operative ospedaliere.

- **id_reparto** (PK): Identificatore univoco del reparto.
- **nome_reparto**: Nome o etichetta del reparto (categorico).
- **specialita**: Specialità clinica del reparto (es. Cardiologia, Neurologia, Oncologia, Pediatria, Pronto Soccorso, Terapia Intensiva, Ortopedia).

2. **Personale («personale»)** Anagrafica dello staff medico e sanitario.

- **id_staff** (PK): Identificatore univoco del membro dello staff.
- **nome**: Nome proprio del membro dello staff.
- **cognome**: Cognome del membro dello staff.
- **ruolo**: Ruolo professionale (es. Infermiere, Medico, Tecnico, Terapista).
- **reparto**: Reparto o area di assegnazione (coerente con le specialità dei reparti).
- **tipo_impiego**: Tipologia di impiego (es. Tempo pieno, Part-time, Contratto).
- **email**: Indirizzo email aziendale dello staff.
- **telefono**: Numero di telefono dello staff.
- **id_licenza**: Identificativo professionale sintetico.
- **data_assunzione**: Data di assunzione.

3. **Assegnazioni («assegnazioni»)** Gestisce l'assegnazione operativa del personale ai reparti.

- **id_assegnazione** (PK): Identificatore univoco dell'assegnazione.
- **id_staff** (FK): Riferimento allo staff (FK -> personale.id_staff).
- **id_reparto** (FK): Riferimento al reparto (FK -> reparti.id_reparto).
- **turno**: Turno di lavoro (es. Giorno, Notte, Sera).

2.4.2 Dominio IoT

4. **Dispositivi («dispositivi»)** Inventario dei dispositivi medici IoT connessi.

- **id_dispositivo** (PK): Identificatore univoco del dispositivo.
- **id_reparto** (FK): Riferimento reparto di utilizzo/allocazione (FK -> reparti.id_reparto).
- **tipo_dispositivo**: Tipo dispositivo (es. ECG, Pulsossimetro, Sfigmomanometro, Termometro).
- **produttore**: Produttore del dispositivo.
- **modello**: Modello o codice del dispositivo.
- **numero_serie**: Numero di serie del dispositivo.
- **stato**: Stato operativo (es. Attivo, Manutenzione, Ritirato).
- **data_acquisto**: Data di acquisto.
- **data_ultima_calibrazione**: Data dell'ultima calibrazione (con vincolo $\geq$ data_acquisto).

5. Parametri Vitali («parametri_vitali») Registro delle misurazioni vitali sparse effettuate dai dispositivi.

- **id_misurazione** (PK): Identificatore della misurazione.
- **id_paziente** (FK): Riferimento paziente (FK -> pazienti.id_paziente).
- **id_dispositivo** (FK): Riferimento dispositivo (FK -> dispositivi.id_dispositivo).
- **data_misurazione**: Data e ora della misurazione.
- **frequenza_cardiaca**: Battiti per minuto (intero).
- **saturazione_ossigeno**: SpO2 in percentuale (intero).
- **pressione_sistolica**: Pressione sistolica (intero).
- **pressione_diastolica**: Pressione diastolica (intero).
- **temperatura_c**: Temperatura corporea in °C (float).
- **frequenza_respiratoria**: Atti respiratori al minuto (intero).
- **glicemia_mg_dl**: Glicemia in mg/dL (intero).

Nota tecnica: I campi dei parametri vitali sono popolati condizionalmente in base al **tipo di dispositivo** associato. Ad esempio, un Termometro valorizzerà esclusivamente il campo **temperatura_c**, lasciando gli altri a **NULL**. Questa scelta rende il dataset realistico e introduce valori mancanti (missing values) che dovranno essere gestiti nelle fasi di analisi.

2.4.3 Dominio EHR

6. Pazienti («pazienti») Anagrafica centrale dei pazienti. *Identificativi:*

- **id_paziente** (PK): Identificatore univoco del paziente.
- **codice_fiscale**: Identificativo nazionale sintetico.

Anagrafica:

- **nome**: Nome.
- **cognome**: Cognome.
- **Sesso**: Sesso biologico (F o M).
- **data_nascita**: Data di nascita.
- **stato_civile**: Stato civile (celibe/nubile, sposato/a, divorziato/a, vedovo/a).
- **lingua_primaria**: Lingua primaria (es. it).
- **gruppo_sanguigno**: Gruppo sanguigno (A+, A-, B+, B-, AB+, AB-, O+, O-).

Contatto e indirizzo:

- **citta**: Città di residenza (trattabile come PII).
- **indirizzo**: Indirizzo stradale.
- **cap**: CAP.
- **paese**: Paese di residenza.
- **email**: Email personale.
- **telefono**: Telefono personale.

Assicurazione:

- **compagnia_assicurativa**: Provider assicurativo.
- **piano_assicurativo**: Piano sottoscritto (basic, standard, premium).
- **id_assicurazione**: Identificativo polizza.

Contatto emergenza:

- **contatto_emergenza_nome**: Nome contatto emergenza.
- **contatto_emergenza_telefono**: Telefono contatto emergenza.

Misure fisiche:

- **altezza_cm**: Altezza in cm.
- **peso_kg**: Peso in kg.

Nota sulla privacy: Diversi campi della tabella «pazienti» (in particolare contatti e indirizzi) costituiscono informazioni identificabili personalmente (PII). La gestione degli accessi e la protezione di questi dati verranno affrontate specificamente nelle sezioni relative alla sicurezza (RBAC e conformità GDPR).

7. Ricoveri («ricoveri») Storico delle ospedalizzazioni.

- **id_ricovero** (PK): Identificatore ricovero.
- **id_paziente** (FK): Riferimento paziente (FK -> pazienti.id_paziente).
- **id_reparto** (FK): Riferimento reparto (FK -> reparti.id_reparto).
- **data_ricovero**: Data e ora ricovero.
- **data_dimissione**: Data e ora dimissione (con vincolo $\geq$ data_ricovero).
- **durata_degenza_giorni**: Durata degenza in giorni (tra 1 e 30).
- **tipo_ricovero**: Tipo di ricovero (Emergenza, Elettivo, Urgente).
- **provenienza_ricovero**: Provenienza (PS, Invio, Trasferimento).
- **esito_dimissione**: Esito alla dimissione (Domicilio, Trasferimento, Riabilitazione, Deceduto).

8. Diagnosi («diagnosi») Dettaglio delle diagnosi associate ai ricoveri.

- **id_diagnosi** (PK): Identificatore diagnosi.
- **id_ricovero** (FK): Riferimento ricovero (FK -> ricoveri.id_ricovero).
- **codice_icd10**: Codice ICD10 (categorico).
- **gravita**: Livello di gravità (bassa, media, alta).

2.5 Relazioni e integrità referenziale

La struttura del dataset si basa su solide relazioni gerarchiche che vengono applicate già nella pipeline di campionamento sintetico e i cui vincoli di integrità (chiavi esterne e domini) sono stati validati:

- `reparti.id_reparto -> assegnazioni.id_reparto`
- `reparti.id_reparto -> dispositivi.id_reparto`
- `reparti.id_reparto -> ricoveri.id_reparto`
- `pazienti.id_paziente -> ricoveri.id_paziente`
- `pazienti.id_paziente -> parametri_vitali.id_paziente`
- `personale.id_staff -> assegnazioni.id_staff`
- `dispositivi.id_dispositivo -> parametri_vitali.id_dispositivo`
- `ricoveri.id_ricovero -> diagnosi.id_ricovero`

Queste relazioni sono il prerequisito fondamentale per il modello Snowflake, permettendo analisi incrociate tra domini diversi.

2.6 Ruoli, permessi e predisposizione per integrazione Snowflake

In questa fase, ho impostato l'infrastruttura di storage cloud su Amazon S3 e definito il modello di sicurezza per la futura integrazione con Snowflake. Ho adottato un approccio architetturale basato su principi di sicurezza enterprise: utilizzo di un bucket completamente privato, autenticazione tramite ruolo IAM (senza credenziali statiche) e applicazione rigorosa del principio del **least privilege**.

Questa configurazione garantisce che la separazione tra lo strato di storage (Data Lake) e lo strato di computazione (Data Warehouse) avvenga in modo sicuro e controllato.

2.6.1 Bucket S3 e struttura interna dei dati

Ho creato un bucket S3 dedicato nella regione AWS **eu-central-1** (Francoforte), denominandolo esplicitamente **healthcare-data-prod-eu**. Il bucket è configurato con il blocco totale dell'accesso pubblico («Block Public Access» attivo).

Ho strutturato i dati internamente utilizzando una gerarchia di cartelle (prefix) che riflette l'organizzazione logica del dataset, facilitando la futura ingestione automatizzata:

```
healthcare-data-prod-eu/  
└─ snowflake/  
   └─ raw/  
      ├── ehr/  
      │   ├── pazienti/  
      │   ├── ricoveri/  
      │   └─ diagnosi/  
      ├── erp/  
      │   ├── reparti/  
      │   ├── personale/  
      │   └─ assegnazioni/  
      └─ iot/  
          ├── dispositivi/  
          └─ parametri_vitali/
```

Codice 1: Struttura delle cartelle nel bucket S3

Ogni directory foglia (es. `pazienti/`) contiene esclusivamente i file dati (Parquet) relativi a quella specifica entità, garantendo omogeneità di schema.

2.6.2 Ruolo IAM dedicato a Snowflake

Per abilitare l'accesso sicuro da parte di Snowflake, ho creato un ruolo IAM (Identity and Access Management) dedicato nel mio account AWS, denominato **snowflake_s3_readonly_role**. Questo ruolo è stato progettato per essere «assumibile» da un'entità esterna fidata (il servizio Snowflake) e non possiede permessi diretti di login o chiavi di accesso.

2.6.3 Policy IAM (Least Privilege)

Al ruolo ho associato una policy IAM custom, denominata **snowflake_s3_raw_read_policy**, che definisce esattamente quali azioni sono permesse. La policy concede esclusivamente i permessi di lettura (`s3:GetObject`) e di listing (`s3:ListBucket`), restringendoli rigorosamente al bucket di produzione e al path specifico dei dati grezzi.

Ecco la definizione della policy applicata:

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": [  
        "s3:ListBucket",  
        "s3:GetObject"  
      ],  
      "Resource": [  
        "arn:aws:s3:::healthcare-data-prod-eu",  
        "arn:aws:s3:::healthcare-data-prod-eu/snowflake/raw/*"  
      ]  
    }  
  ]  
}
```

Codice 2: Policy IAM (least privilege)

Questa configurazione impedisce qualsiasi operazione di scrittura o cancellazione e nega l'accesso a qualsiasi altra risorsa AWS non esplicitamente indicata.

2.6.4 Trust policy del ruolo IAM (predisposta con placeholder)

Per completare la configurazione di sicurezza, il ruolo IAM necessita di una «Trust Relationship» che autorizzi specificamente l'account Snowflake ad assumerlo. Poiché l'account Snowflake non è ancora stato collegato, ho predisposto la Trust Policy utilizzando dei placeholder espliciti per i valori che verranno generati lato Snowflake:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "<SNOWFLAKE_IAM_USER_ARN>"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "<SNOWFLAKE_EXTERNAL_ID>"
        }
      }
    }
  ]
}
```

Codice 3: Trust Policy del ruolo IAM (con placeholder)

I valori <SNOWFLAKE_IAM_USER_ARN> (l'identità utente IAM di Snowflake) e <SNOWFLAKE_EXTERNAL_ID> (un identificativo univoco per la sicurezza cross-account) verranno recuperati eseguendo il comando `DESC INTEGRATION` su Snowflake e aggiornati in questa policy in un secondo momento.

3 Architettura logica su Snowflake

3.1 Visione d'insieme

In questa sezione documento la configurazione dell'ambiente Snowflake che ho realizzato.

Utilizzando un account trial di 30 giorni, ho implementato un'architettura che separa nettamente le risorse di calcolo da quelle di storage, sfruttando la natura nativa cloud della piattaforma.

L'obiettivo è passare da un disegno logico a un'implementazione fisica verificabile, preparando il terreno per l'ingestione e la trasformazione dei dati.

3.2 Creazione del database principale

Come primo passo, ho creato il container logico di alto livello per ospitare tutti gli oggetti del progetto. Ho scelto il nome `HEALTHCARE_DW` per identificare chiaramente l'ambito di dominio.

```
CREATE DATABASE HEALTHCARE_DW;
```

Codice 4: Creazione del database `HEALTHCARE_DW`

Subito dopo l'esecuzione, ho verificato la corretta creazione del database interrogando i metadati di sistema:

```
SHOW DATABASES LIKE 'HEALTHCARE_DW';
```

Codice 5: Verifica creazione database

Per assicurarmi che tutti i comandi successivi vengano eseguiti nel contesto corretto, ho impostato il database attivo:

```
USE DATABASE HEALTHCARE_DW;
```

Codice 6: Impostazione del database attivo

3.3 Implementazione del Data Layering (Schemi)

Per riflettere la stratificazione dei dati definita durante l'analisi, ho suddiviso il database in quattro schemi distinti. Questa organizzazione logica mi permette di governare il ciclo di vita del dato:

1. **RAW**: Punto di atterraggio dei dati grezzi provenienti da S3.
2. **CURATED**: Livello intermedio per pulizia e normalizzazione.
3. **ANALYTICS**: Livello finale ottimizzato per le query di analisi.
4. **PIPELINE**: schema tecnico dedicato all'orchestrazione che ospiterà le procedure stored e i task chain, separando la logica di controllo dai dati.

Ho eseguito il seguente script DDL:

```
CREATE SCHEMA HEALTHCARE_DW.RAW;  
CREATE SCHEMA HEALTHCARE_DW.CURATED;  
CREATE SCHEMA HEALTHCARE_DW.ANALYTICS;  
CREATE SCHEMA HEALTHCARE_DW.PIPELINE;
```

Codice 7: Creazione degli schemi (Data Layering)

Per confermare la struttura, ho listato gli schemi presenti nel database:

```
SHOW SCHEMAS IN DATABASE HEALTHCARE_DW;
```

Codice 8: Verifica creazione schemi

3.4 Configurazione delle risorse di calcolo (Virtual Warehouse)

Per garantire prestazioni ottimali e isolamento dei carichi di lavoro, ho adottato una strategia multi-warehouse. In Snowflake, separare le risorse di calcolo impedisce che operazioni diverse (come un caricamento massivo dati) competano per le stesse risorse di una query di analisi, rallentandola. Ogni warehouse opera in modo indipendente, permettendo di scalare verticalmente o orizzontalmente specifici carichi di lavoro senza impattare le prestazioni delle altre attività.

Ho quindi definito tre warehouse distinti, ciascuno dimensionato in base al compito specifico.

3.4.1 Warehouse per Ingestion

Il primo warehouse è dedicato alle operazioni di caricamento dati (ETL/ELT). Ho scelto una taglia **XSMALL** per contenere i costi durante le fasi di copia e trasformazione iniziale.

```
CREATE WAREHOUSE WH_INGEST  
WITH WAREHOUSE_SIZE = 'XSMALL'  
AUTO_SUSPEND = 60  
AUTO_RESUME = TRUE  
INITIALLY_SUSPENDED = TRUE;
```

Codice 9: Definizione Warehouse per Ingestion

3.4.2 Warehouse per Operazioni (Consultazione)

Per le query frequenti a bassa latenza e l'interrogazione puntuale dei dati, ho creato un warehouse separato. Questo garantisce che le attività di consultazione non vengano bloccate dai processi di caricamento.

```
CREATE WAREHOUSE WH_OPERATIONS  
WITH WAREHOUSE_SIZE = 'XSMALL'  
AUTO_SUSPEND = 60  
AUTO_RESUME = TRUE  
INITIALLY_SUSPENDED = TRUE;
```

Codice 10: Definizione Warehouse per Operazioni

3.4.3 Warehouse per Analisi (Analytics)

Infine, ho configurato un warehouse dedicato ai carichi di lavoro analitici complessi (query analitiche su dataset aggregati e calcolo di KPI complessi). Per questo carico ho selezionato una taglia **Small** (doppia potenza rispetto a XSMALL), necessaria per gestire query più onerose su volumi di dati aggregati.

```
CREATE WAREHOUSE WH_ANALYTICS
  WITH WAREHOUSE_SIZE = 'SMALL'
  AUTO_SUSPEND = 120
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;
```

Codice 11: Definizione Warehouse per Analytics

3.5 Riepilogo assetto architetturale

Al termine di queste operazioni, l'ambiente Snowflake è configurato come segue:

- **Database:** HEALTHCARE_DW
- **Struttura Schemi:**
 - RAW → Ingestione da Stage
 - CURATED → Trasformazioni
 - ANALYTICS → Consumo dati
 - PIPELINE → Orchestrazione
- **Compute:**
 - WH_INGEST (XSMALL) → Caricamento dati
 - WH_OPERATIONS (XSMALL) → Query operative
 - WH_ANALYTICS (Small) → Analisi avanzata

Questa configurazione costituisce le fondamenta per l'implementazione del modello dati fisico che descriverò nella prossima sezione.

4 Progettazione del modello dati

4.1 Obiettivo del modello e criteri di progettazione

In questa sezione definisco il modello dati fisico su Snowflake, traducendo i requisiti analitici in strutture tabellari concrete. L'architettura segue il pattern a livelli (RAW, CURATED, ANALYTICS) già predisposto.

4.2 Layer RAW – Tabelle di atterraggio (Mirror)

Il layer RAW funge da area di staging. Le tabelle qui definite rispecchiano la struttura del dizionario dati e sono compatibili con i Parquet generati presenti nel Data Lake su S3. Non applico trasformazioni né vincoli di integrità in questa fase: l'obiettivo è garantire un atterraggio veloce e sicuro del dato grezzo.

Tutti i tipi di dato sono stati mappati dal dizionario dati ai tipi nativi di Snowflake (VARCHAR, DATE, TIMESTAMP_NTZ, NUMBER, FLOAT).

4.2.1 RAW – Dominio ERP (Enterprise Resource Planning)

DDL RAW: tabelle del dominio ERP

```
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.RAW;
```

```
-- 1. Reparti
CREATE OR REPLACE TABLE REPARTI (
  ID_REPARTO VARCHAR,
```

```

    NOME_REPARTO VARCHAR,
    SPECIALITA VARCHAR
) COMMENT = 'Mirror raw dati reparti (ERP)';

-- 2. Personale
CREATE OR REPLACE TABLE PERSONALE (
    ID_STAFF VARCHAR,
    NOME VARCHAR, --PII
    COGNOME VARCHAR, --PII
    RUOLO VARCHAR,
    REPARTO VARCHAR,
    TIPO_IMPIEGO VARCHAR,
    EMAIL VARCHAR, --PII
    TELEFONO VARCHAR, --PII
    ID_LICENZA VARCHAR,
    DATA_ASSUNZIONE DATE
) COMMENT = 'Mirror raw anagrafica personale (ERP)';

-- 3. Assegnazioni
CREATE OR REPLACE TABLE ASSEGNAZIONI (
    ID_ASSEGNAZIONE VARCHAR,
    ID_STAFF VARCHAR,
    ID_REPARTO VARCHAR,
    TURNO VARCHAR
) COMMENT = 'Mirror raw assegnazioni staff-reparto (ERP)';

```

4.2.2 RAW – Dominio IoT (Internet of Things)

DDL RAW: tabelle del dominio IoT

```

USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.RAW;

-- 4. Dispositivi
CREATE OR REPLACE TABLE DISPOSITIVI (
    ID_DISPOSITIVO VARCHAR,
    ID_REPARTO VARCHAR,
    TIPO_DISPOSITIVO VARCHAR,
    PRODUTTORE VARCHAR,
    MODELLO VARCHAR,
    NUMERO_SERIE VARCHAR,
    STATO VARCHAR,
    DATA_ACQUISTO DATE,
    DATA_ULTIMA_CALIBRAZIONE DATE
) COMMENT = 'Mirror raw dispositivi medici (IoT)';

-- 5. Parametri Vitali
CREATE OR REPLACE TABLE PARAMETRI_VITALI (
    ID_MISURAZIONE VARCHAR,
    ID_PAZIENTE VARCHAR,
    ID_DISPOSITIVO VARCHAR,
    DATA_MISURAZIONE TIMESTAMP_NTZ,
    FREQUENZA_CARDIACA NUMBER(3,0),
    SATURAZIONE_OSSIGENO NUMBER(3,0),
    PRESSIONE_SISTOLICA NUMBER(3,0),
    PRESSIONE_DIASTOLICA NUMBER(3,0),
    TEMPERATURA_C FLOAT,
    FREQUENZA_RESPIRATORIA NUMBER(3,0),
    GLICEMIA_MG_DL NUMBER(3,0)
) COMMENT = 'Mirror raw telemetria vitale (IoT)';

```

4.2.3 RAW – Dominio EHR (Electronic Health Record)

DDL RAW: tabelle del dominio EHR

```
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.RAW;

-- 6. Pazienti
CREATE OR REPLACE TABLE PAZIENTI (
  ID_PAZIENTE VARCHAR,
  CODICE_FISCALE VARCHAR, --PII
  NOME VARCHAR, --PII
  COGNOME VARCHAR, --PII
  SESSO VARCHAR,
  DATA_NASCITA DATE, --PII
  STATO_CIVILE VARCHAR,
  LINGUA_PRIMARIA VARCHAR,
  GRUPPO_SANGUIGNO VARCHAR,
  CITTA VARCHAR, --PII
  INDIRIZZO VARCHAR, --PII
  CAP VARCHAR,
  PAESE VARCHAR,
  EMAIL VARCHAR, --PII
  TELEFONO VARCHAR, --PII
  COMPAGNIA_ASSICURATIVA VARCHAR,
  PIANO_ASSICURATIVO VARCHAR,
  ID_ASSICURAZIONE VARCHAR,
  CONTATTO_EMERGENZA_NOME VARCHAR, --PII
  CONTATTO_EMERGENZA_TELEFONO VARCHAR, --PII
  ALTEZZA_CM NUMBER(3,0),
  PESO_KG NUMBER(5,2)
) COMMENT = 'Mirror raw anagrafica pazienti (EHR)';

-- 7. Ricoveri
CREATE OR REPLACE TABLE RICOVERI (
  ID_RICOVERO VARCHAR,
  ID_PAZIENTE VARCHAR,
  ID_REPARTO VARCHAR,
  DATA_RICOVERO TIMESTAMP_NTZ,
  DATA_DIMISSIONE TIMESTAMP_NTZ,
  DURATA_DEGENZA_GIORNI NUMBER(3,0),
  TIPO_RICOVERO VARCHAR,
  PROVENIENZA_RICOVERO VARCHAR,
  ESITO_DIMISSIONE VARCHAR
) COMMENT = 'Mirror raw ricoveri ospedalieri (EHR)';

-- 8. Diagnosi
CREATE OR REPLACE TABLE DIAGNOSI (
  ID_DIAGNOSI VARCHAR,
  ID_RICOVERO VARCHAR,
  CODICE_ICD10 VARCHAR,
  GRAVITA VARCHAR
) COMMENT = 'Mirror raw diagnosi cliniche (EHR)';
```

Nota: rappresento PESO_KG con due decimali.

4.3 Layer CURATED – Standardizzazione e qualità

Nel layer CURATED, consolido i dati applicando regole di standardizzazione e definendo vincoli relazionali.

La strategia adottata per la creazione delle tabelle prevede di ereditare la struttura dal layer RAW (LIKE RAW...) per massimizzare la coerenza, aggiungendo poi esplicitamente le chiavi primarie e esterne.

4.3.1 CURATED – Tabelle Anagrafiche e Strutturali (ERP Key)

DDL CURATED: Anagrafiche e ERP

```
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.CURATED;

-- Creazione e vincoli: Reparti e Personale
CREATE OR REPLACE TABLE REPARTI LIKE HEALTHCARE_DW.RAW.REPARTI;
ALTER TABLE REPARTI ADD PRIMARY KEY (ID_REPARTO);

CREATE OR REPLACE TABLE PERSONALE LIKE HEALTHCARE_DW.RAW.PERSONALE;
ALTER TABLE PERSONALE ADD PRIMARY KEY (ID_STAFF);

-- Creazione e vincoli: Assegnazioni
CREATE OR REPLACE TABLE ASSEGNAZIONI LIKE HEALTHCARE_DW.RAW.ASSEGNAZIONI;
ALTER TABLE ASSEGNAZIONI ADD PRIMARY KEY (ID_ASSEGNAZIONE);
ALTER TABLE ASSEGNAZIONI ADD CONSTRAINT FK_ASS_STAFF FOREIGN KEY (ID_STAFF) REFERENCES
PERSONALE(ID_STAFF);
ALTER TABLE ASSEGNAZIONI ADD CONSTRAINT FK_ASS_REP FOREIGN KEY (ID_REPARTO) REFERENCES
REPARTI(ID_REPARTO);
```

4.3.2 CURATED – Dispositivi e Parametri Vitali (IoT)

DDL CURATED: IoT e Telemetria

```
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.CURATED;

-- Creazione e vincoli: Dispositivi
CREATE OR REPLACE TABLE DISPOSITIVI LIKE HEALTHCARE_DW.RAW.DISPOSITIVI;
ALTER TABLE DISPOSITIVI ADD PRIMARY KEY (ID_DISPOSITIVO);
ALTER TABLE DISPOSITIVI ADD CONSTRAINT FK_DEV_REP FOREIGN KEY (ID_REPARTO) REFERENCES
REPARTI(ID_REPARTO);

-- Creazione e vincoli: Parametri Vitali
CREATE OR REPLACE TABLE PARAMETRI_VITALI LIKE HEALTHCARE_DW.RAW.PARAMETRI_VITALI;
ALTER TABLE PARAMETRI_VITALI ADD PRIMARY KEY (ID_MISURAZIONE);
-- FK note: puntano a Pazienti (definito nel blocco EHR) e Dispositivi
ALTER TABLE PARAMETRI_VITALI ADD CONSTRAINT FK_VIT_DEV FOREIGN KEY (ID_DISPOSITIVO)
REFERENCES DISPOSITIVI(ID_DISPOSITIVO);
-- (FK verso Pazienti viene applicata logicamente, ordine di creazione permettendo)
```

4.3.3 CURATED – Pazienti e Ricoveri (EHR)

DDL CURATED: Pazienti e Cartella Clinica

```
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.CURATED;

-- Creazione e vincoli: Pazienti
CREATE OR REPLACE TABLE PAZIENTI LIKE HEALTHCARE_DW.RAW.PAZIENTI;
ALTER TABLE PAZIENTI ADD PRIMARY KEY (ID_PAZIENTE);

-- Creazione e vincoli: Ricoveri
CREATE OR REPLACE TABLE RICOVERI LIKE HEALTHCARE_DW.RAW.RICOVERI;
ALTER TABLE RICOVERI ADD PRIMARY KEY (ID_RICOVERO);
ALTER TABLE RICOVERI ADD CONSTRAINT FK_RIC_PAZ FOREIGN KEY (ID_PAZIENTE) REFERENCES
```

```

PAZIENTI(ID_PAZIENTE);
ALTER TABLE RICOVERI ADD CONSTRAINT FK_RIC_REP FOREIGN KEY (ID_REPARTO) REFERENCES
REPARTI(ID_REPARTO);

-- Creazione e vincoli: Diagnosi
CREATE OR REPLACE TABLE DIAGNOSI LIKE HEALTHCARE_DW.RAW.DIAGNOSI;
ALTER TABLE DIAGNOSI ADD PRIMARY KEY (ID_DIAGNOSI);
ALTER TABLE DIAGNOSI ADD CONSTRAINT FK_DIAG_RIC FOREIGN KEY (ID_RICOVERO) REFERENCES
RICOVERI(ID_RICOVERO);

-- completamento relazioni circolari o dipendenti
ALTER TABLE PARAMETRI_VITALI ADD CONSTRAINT FK_VIT_PAZ FOREIGN KEY (ID_PAZIENTE)
REFERENCES PAZIENTI(ID_PAZIENTE);

```

4.3.4 CURATED – Gestione Anomalie (Quarantena)

Le strutture *_QUARANTENA vengono predisposte per ospitare i record che falliscono le validazioni di qualità o l'integrità referenziale durante l'esecuzione delle pipeline. Mantengono la struttura delle tabelle RAW per facilitare il debug e l'analisi delle anomalie (Data Quality) senza interrompere il flusso di caricamento.

DDL CURATED: Tabelle di Quarantena

```

USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.CURATED;

-- 1. Quarantena Ricoveri (es. paziente non trovato o errori di formato)
CREATE TABLE IF NOT EXISTS RICOVERI_QUARANTENA LIKE HEALTHCARE_DW.RAW.RICOVERI;

-- 2. Quarantena Parametri Vitali (es. valori fuori soglia o device dismesso)
CREATE TABLE IF NOT EXISTS PARAMETRI_VITALI_QUARANTENA LIKE
HEALTHCARE_DW.RAW.PARAMETRI_VITALI;

```

4.4 Layer ANALYTICS – Modello dimensionale

Il livello ANALYTICS è modellato secondo i principi dello **Star Schema** per facilitare l'analisi tramite strumenti di BI.

Per mantenere il modello snello e coerente con i dati sintetici, ho utilizzato le **Business Key** originali come chiavi primarie delle dimensioni, evitando di rigenerare chiavi surrogate autoincrementali che avrebbero introdotto complessità non necessaria in questo scenario.

Schema progettato:

- **Dimensioni (DIM):** DIM_PAZIENTE, DIM_REPARTO, DIM_DISPOSITIVO.
- **Fatti (FACT):** FACT_RICOVERI (eventi clinici), FACT_MISURAZIONI (eventi IoT).
- **Bridge:** La tabella DIAGNOSI viene trattata come una tabella ponte (BRIDGE_DIAGNOSI) che lega diagnosi multiple al singolo ricovero.

La considero un evento dipendente dal ricovero (1:N) e non una dimensione autonoma perché non è un attributo descrittivo stabile; così mantengo lo schema snello e coerente col dataset sintetico.

4.4.1 ANALYTICS – Dimensioni

DDL ANALYTICS: Definizione delle Dimensioni

```

USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.ANALYTICS;

-- Dimensione Tempo
CREATE OR REPLACE TABLE DIM_TEMPO (
    DATA DATE PRIMARY KEY,

```

```

        ANNO INT,
        MESE INT,
        NOME_MESE VARCHAR,
        GIORNO_SETTIMANA VARCHAR,
        TRIMESTRE INT
    );

-- Dimensione Reparto
CREATE OR REPLACE TABLE DIM_REPARTO (
    ID_REPARTO VARCHAR PRIMARY KEY,
    NOME_REPARTO VARCHAR,
    SPECIALITA VARCHAR
);

-- Dimensione Dispositivo
CREATE OR REPLACE TABLE DIM_DISPOSITIVO (
    ID_DISPOSITIVO VARCHAR PRIMARY KEY,
    TIPO_DISPOSITIVO VARCHAR,
    MODELLO VARCHAR,
    PRODUTTORE VARCHAR
);

-- Dimensione Paziente
CREATE OR REPLACE TABLE DIM_PAZIENTE (
    ID_PAZIENTE VARCHAR PRIMARY KEY,
    SESSO VARCHAR,
    DATA_NASCITA DATE, --PII
    CITTA VARCHAR, --PII
    GRUPPO_SANGUIGNO VARCHAR,
    PIANO_ASSICURATIVO VARCHAR
);

```

4.4.2 ANALYTICS – Tabelle dei Fatti e Bridge

DDL ANALYTICS: Fact Tables e Bridge

```

USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.ANALYTICS;

-- Fact Table: Ricoveri
CREATE OR REPLACE TABLE FACT_RICOVERI (
    ID_RICOVERO VARCHAR PRIMARY KEY,
    -- FK verso Dimensioni (Business Keys)
    ID_PAZIENTE VARCHAR REFERENCES DIM_PAZIENTE(ID_PAZIENTE),
    ID_REPARTO VARCHAR REFERENCES DIM_REPARTO(ID_REPARTO),
    TS_RICOVERO TIMESTAMP_NTZ,
    TS_DIMISSIONE TIMESTAMP_NTZ,

    -- Metriche
    DURATA_DEGENZA_GIORNI NUMBER(3,0),
    TIPO_RICOVERO VARCHAR,
    ESITO_DIMISSIONE VARCHAR
);

-- Fact Table: Misurazioni (Wide)
CREATE OR REPLACE TABLE FACT_MISURAZIONI (
    ID_MISURAZIONE VARCHAR PRIMARY KEY,
    -- FK verso Dimensioni
    ID_PAZIENTE VARCHAR REFERENCES DIM_PAZIENTE(ID_PAZIENTE),
    ID_DISPOSITIVO VARCHAR REFERENCES DIM_DISPOSITIVO(ID_DISPOSITIVO),

```



```

TS_MISURAZIONE TIMESTAMP_NTZ,
-- Campo derivato da TS_MISURAZIONE (solo per query veloci per fascia oraria)
ORA_GIORNO NUMBER(2,0),

-- Metriche
FREQUENZA_CARDIACA NUMBER(3,0),
SATURAZIONE_OSSIGENO NUMBER(3,0),
PRESSIONE_SISTOLICA NUMBER(3,0),
PRESSIONE_DIASTOLICA NUMBER(3,0),
TEMPERATURA_C FLOAT,
GLICEMIA_MG_DL NUMBER(3,0)
);

-- Bridge Table: Diagnosi (1:N su Ricoveri)
CREATE OR REPLACE TABLE BRIDGE_DIAGNOSI (
    ID_DIAGNOSI VARCHAR PRIMARY KEY,
    ID_RICOVERO VARCHAR REFERENCES FACT_RICOVERI(ID_RICOVERO),
    CODICE_ICD10 VARCHAR,
    GRAVITA VARCHAR
);

```

4.5 Conclusioni

In questa sezione ho definito l'architettura informativa su Snowflake, ora allineata e coerente con il dizionario dati.

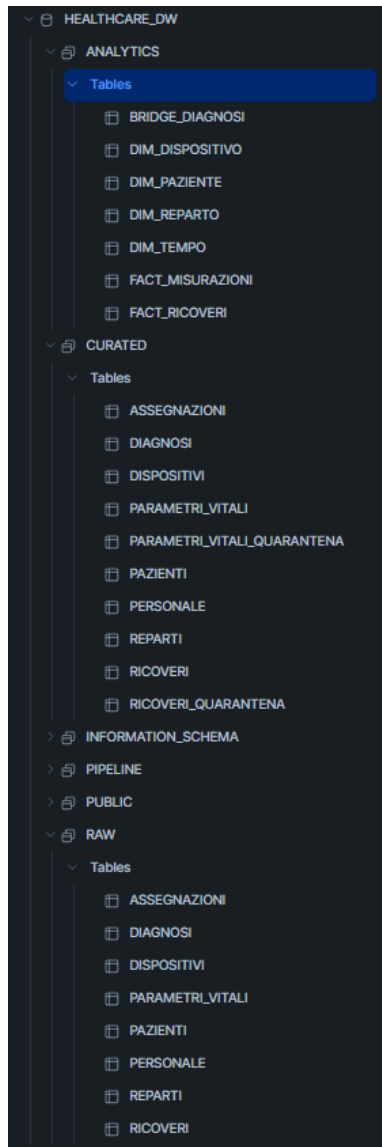


Figura 1: Screenshot: vista UI degli schemi e delle tabelle nel database HEALTHCARE_DW

5 Sicurezza e conformità normativa

In continuità con la progettazione del modello dati, la sicurezza è stata definita come una **proprietà del ciclo di vita del dato** e non come un insieme di regole astratte. La separazione RAW → CURATED → ANALYTICS non rappresenta solo una scelta di organizzazione logica, ma diventa la base per applicare controlli di accesso e protezioni diverse in funzione di:

- sensibilità del contenuto (presenza di PII e dati sanitari)
- scopo del layer (atterraggio, standardizzazione, consumo)
- responsabilità operative (ingestion/ELT vs analisi)

5.1 7.1 Principi di sicurezza applicati al modello

La strategia di sicurezza segue direttamente la stratificazione a tre layer definita nel data warehouse HEALTHCARE_DW.

- **Layer RAW (staging / mirror):** contiene i dati così come arrivano dalle sorgenti, inclusa la componente più ricca di PII nella tabella

RAW.PAZIENTI (es. CODICE_FISCALE, contatti, indirizzo, email, telefono). Per questo motivo l'accesso a RAW è ristretto e destinato esclusivamente a chi gestisce l'ingestione e le trasformazioni.

- **Layer CURATED (standardizzazione e qualità):** rappresenta il punto in cui applico le regole di qualità e le relazioni (PK/FK **informational**, non enforced).

Essendo che l'integrità è demandata alla pipeline ELT, limito i privilegi di scrittura a ruoli tecnici: in questo modo riduco il rischio di scritture manuali non tracciate che potrebbero introdurre incoerenze rispetto alle validazioni.

- **Layer ANALYTICS (consumo):** ospita il modello dimensionale (DIM_*, FACT_*) progettato per BI. È il livello in cui operano gli utenti di business;

di conseguenza, l'accesso per analisi avviene qui e non su RAW. Anche in ANALYTICS sono presenti attributi che possono costituire PII o quasi-identificatori nella DIM_PAZIENTE (es. DATA_NASCITA, CITTA), quindi il consumo viene protetto con **masking policy** applicate a colonne specifiche.

5.2 7.2 Strategia RBAC su Snowflake

Ho implementato il controllo accessi tramite **RBAC nativo Snowflake**, definendo ruoli coerenti con le responsabilità sul ciclo di vita del dato. L'obiettivo è che le pipeline ELT operino con un ruolo tecnico dedicato e che gli utenti analitici lavorino esclusivamente sul layer di consumo.

Ruoli definiti:

- **ROLE_DATA_ENGINEER:** ingestione e trasformazioni (scrittura su RAW/CURATED, pubblicazione su ANALYTICS).
- **ROLE_DATA_ANALYST:** consultazione e analisi su ANALYTICS (e, dove necessario, CURATED in sola lettura).
- **ROLE_COMPLIANCE_OFFICER:** verifica e auditing; **visibilità completa ai fini di controllo** (lettura),

inclusa la visibilità non mascherata degli attributi PII dove previsto dalle policy, senza privilegi di scrittura.

DDL Snowflake: creazione ruoli

```
USE ROLE SECURITYADMIN;
```

```
CREATE ROLE IF NOT EXISTS ROLE_DATA_ENGINEER;
CREATE ROLE IF NOT EXISTS ROLE_DATA_ANALYST;
CREATE ROLE IF NOT EXISTS ROLE_COMPLIANCE_OFFICER;
```

5.2.1 Gerarchia dei ruoli

Per semplificare la gestione e garantire che chi controlla abbia sempre almeno la stessa visibilità di chi analizza, ho stabilito una gerarchia esplicita. Il **ROLE_COMPLIANCE_OFFICER** eredita il **ROLE_DATA_ANALYST**: questo assicura che qualsiasi oggetto esposto per l'analisi sia automaticamente visibile (e auditabile) dal Compliance Officer senza necessità di doppi privilegi.

```
USE ROLE SECURITYADMIN;
```

```
-- Il Compliance Officer "è anche" un Data Analyst (vede ciò che vede l'analista)
GRANT ROLE ROLE_DATA_ANALYST TO ROLE ROLE_COMPLIANCE_OFFICER;
```

5.2.2 Assegnazione ruoli all'utente di lab

Per facilitare il testing e lo switch tra i diversi profili durante la validazione, assegno esplicitamente i tre ruoli definiti all'utente principale (LAB_USER).

```
USE ROLE SECURITYADMIN;
```

```
GRANT ROLE ROLE_DATA_ENGINEER TO USER LAB_USER;
```

```
GRANT ROLE ROLE_DATA_ANALYST TO USER LAB_USER;
GRANT ROLE ROLE_COMPLIANCE_OFFICER TO USER LAB_USER;
```

5.3 7.3 Assegnazione privilegi per layer

La matrice dei privilegi è costruita per schema, riflettendo la funzione di ciascun layer. In tutti i casi, per poter operare su un oggetto Snowflake è necessario garantire **USAGE** sul database e sullo schema, oltre ai privilegi sulle tabelle.

5.3.1 Privilegi sui Virtual Warehouse (coerenza con Sezione 5)

In continuità con la strategia multi-warehouse (WH_INGEST, WH_OPERATIONS, WH_ANALYTICS), ho separato i privilegi di calcolo in modo che i ruoli utilizzino solo le risorse necessarie al proprio carico di lavoro. Questo isolamento riduce la possibilità di interferenza tra ingestion/ELT e query analitiche e rende più governabile il consumo computazionale.

```
USE ROLE SECURITYADMIN;
```

```
GRANT USAGE ON WAREHOUSE WH_INGEST TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON WAREHOUSE WH_ANALYTICS TO ROLE ROLE_DATA_ENGINEER;
```

```
GRANT USAGE ON WAREHOUSE WH_ANALYTICS TO ROLE ROLE_DATA_ANALYST;
-- Il ROLE_COMPLIANCE_OFFICER eredita USAGE su WH_ANALYTICS
```

Prerequisiti comuni (accesso al database)

```
USE ROLE SECURITYADMIN;
```

```
GRANT USAGE ON DATABASE HEALTHCARE_DW TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON DATABASE HEALTHCARE_DW TO ROLE ROLE_DATA_ANALYST;
-- Il ROLE_COMPLIANCE_OFFICER eredita USAGE sul Database
```

Nel layer **RAW** autorizzo la scrittura solo al ruolo tecnico; gli analisti non hanno accesso diretto alle tabelle mirror.

```
USE ROLE SECURITYADMIN;
```

```
-- Accesso allo schema
```

```
GRANT USAGE ON SCHEMA HEALTHCARE_DW.RAW TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON SCHEMA HEALTHCARE_DW.RAW TO ROLE ROLE_COMPLIANCE_OFFICER;
```

```
-- Privilegi sulle tabelle esistenti
```

```
GRANT SELECT, INSERT ON ALL TABLES IN SCHEMA HEALTHCARE_DW.RAW TO ROLE ROLE_DATA_ENGINEER;
GRANT SELECT ON ALL TABLES IN SCHEMA HEALTHCARE_DW.RAW TO ROLE ROLE_COMPLIANCE_OFFICER;
```

```
-- Coerenza nel tempo (oggetti futuri)
```

```
GRANT SELECT, INSERT ON FUTURE TABLES IN SCHEMA HEALTHCARE_DW.RAW TO ROLE
ROLE_DATA_ENGINEER;
GRANT SELECT ON FUTURE TABLES IN SCHEMA HEALTHCARE_DW.RAW TO ROLE ROLE_COMPLIANCE_OFFICER;
```

Nota: l'assenza di **GRANT USAGE** sullo schema **RAW** per **ROLE_DATA_ANALYST** impedisce l'esplorazione diretta dei dati grezzi (inclusa **RAW.PAZIENTI**).

5.3.2 CURATED

Nel layer **CURATED** il **ROLE_DATA_ENGINEER** deve poter effettuare trasformazioni e pubblicazioni (tipicamente tramite **INSERT** e operazioni di upsert/merge). Il **ROLE_DATA_ANALYST** accede in sola lettura per verifiche o analisi intermedie sulla qualità dei dati (inclusa la revisione delle anomalie nelle tabelle di quarantena).

```
USE ROLE SECURITYADMIN;
```

```

GRANT USAGE ON SCHEMA HEALTHCARE_DW.CURATED TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON SCHEMA HEALTHCARE_DW.CURATED TO ROLE ROLE_DATA_ANALYST;

-- Data Engineer: trasformazioni, caricamenti e gestione quarantena (R/W) inclusa TRUNCATE
GRANT SELECT, INSERT, UPDATE, DELETE, TRUNCATE ON ALL TABLES IN SCHEMA
HEALTHCARE_DW.CURATED TO ROLE ROLE_DATA_ENGINEER;
GRANT SELECT, INSERT, UPDATE, DELETE, TRUNCATE ON FUTURE TABLES IN SCHEMA
HEALTHCARE_DW.CURATED TO ROLE ROLE_DATA_ENGINEER;

-- Analyst: sola lettura per audit e data quality verify (ereditata anche da Compliance)
GRANT SELECT ON ALL TABLES IN SCHEMA HEALTHCARE_DW.CURATED TO ROLE ROLE_DATA_ANALYST;
GRANT SELECT ON FUTURE TABLES IN SCHEMA HEALTHCARE_DW.CURATED TO ROLE ROLE_DATA_ANALYST;

```

5.3.3 ANALYTICS

ANALYTICS è il layer di consumo: il ROLE_DATA_ANALYST lavora qui in sola lettura sul modello dimensionale (DIM_*, FACT_* come DIM_PAZIENTE, FACT_RICOVERI). Il ROLE_COMPLIANCE_OFFICER dispone di visibilità completa per controlli e verifiche.

```
USE ROLE SECURITYADMIN;
```

```

GRANT USAGE ON SCHEMA HEALTHCARE_DW.ANALYTICS TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON SCHEMA HEALTHCARE_DW.ANALYTICS TO ROLE ROLE_DATA_ANALYST;

-- Analyst: consumo (SELECT) - Il Compliance Officer eredita questi privilegi
GRANT SELECT ON ALL TABLES IN SCHEMA HEALTHCARE_DW.ANALYTICS TO ROLE ROLE_DATA_ANALYST;
GRANT SELECT ON FUTURE TABLES IN SCHEMA HEALTHCARE_DW.ANALYTICS TO ROLE ROLE_DATA_ANALYST;

-- Data Engineer: pubblicazione e manutenzione delle tabelle del data mart
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA HEALTHCARE_DW.ANALYTICS TO
ROLE ROLE_DATA_ENGINEER;
GRANT SELECT, INSERT, UPDATE, DELETE ON FUTURE TABLES IN SCHEMA HEALTHCARE_DW.ANALYTICS TO
ROLE ROLE_DATA_ENGINEER;

```

5.3.4 PIPELINE

Lo schema PIPELINE ospita la logica di orchestrazione (Stored Procedures, Task, Streams). Trattandosi di uno schema puramente tecnico e operativo, l'accesso è strettamente riservato al Data Engineer, con visibilità di audit per il Compliance Officer.

```
USE ROLE SECURITYADMIN;
```

```

GRANT USAGE ON SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE ROLE_COMPLIANCE_OFFICER;

-- Data Engineer: Creazione e gestione oggetti di orchestrazione
GRANT CREATE TABLE, CREATE VIEW, CREATE PROCEDURE, CREATE TASK, CREATE STREAM
ON SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE ROLE_DATA_ENGINEER;

-- Privilegi operativi su oggetti di supporto (es. tabelle di log)
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA HEALTHCARE_DW.PIPELINE TO
ROLE ROLE_DATA_ENGINEER;
GRANT SELECT, INSERT, UPDATE, DELETE ON FUTURE TABLES IN SCHEMA HEALTHCARE_DW.PIPELINE TO
ROLE ROLE_DATA_ENGINEER;

-- Privilegi per esecuzione e monitoraggio flussi
GRANT OPERATE, MONITOR ON ALL TASKS IN SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE
ROLE_DATA_ENGINEER;
GRANT OPERATE, MONITOR ON FUTURE TASKS IN SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE
ROLE_DATA_ENGINEER;

```

```
GRANT USAGE ON ALL PROCEDURES IN SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE
ROLE_DATA_ENGINEER;
GRANT USAGE ON FUTURE PROCEDURES IN SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE
ROLE_DATA_ENGINEER;
```

```
-- Compliance: sola lettura per audit
GRANT SELECT ON ALL TABLES IN SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE
ROLE_COMPLIANCE_OFFICER;
GRANT SELECT ON FUTURE TABLES IN SCHEMA HEALTHCARE_DW.PIPELINE TO ROLE
ROLE_COMPLIANCE_OFFICER;
```

5.4 7.4 Protezione dei dati sensibili (PII)

L'analisi del modello dati conferma che gli identificatori diretti (NOME, COGNOME, CODICE_FISCALE, EMAIL, TELEFONO) sono stati strutturalmente esclusi dal layer ANALYTICS (Privacy by Design).

Tuttavia, la tabella DIM_PAZIENTE mantiene attributi **quasi-identificatori** che richiedono protezione specifica come identificato nel modello dati (COMMENT 'PII'):

- CITTA: rischio di profilazione geografica (granularità troppo fine).
- DATA_NASCITA: rischio di re-identificazione se incrociata con altre fonti.

Per mitigare questi rischi residui senza bloccare l'accesso al dato aggregato, utilizzo le **Masking Policy** dinamiche di Snowflake. La policy viene applicata a livello di colonna e valuta il ruolo a runtime:

- **Analisti (ROLE_DATA_ANALYST)**: vedono il dato oscurato (** per stringhe, 1900-01-01 per date) in modo da proteggere la privacy individuale mantenendo la struttura tabellare.
- **Compliance & Tecnici**: vedono il dato in chiaro per validazione e auditing.

DDL: Definizione e applicazione delle Masking Policy PII

```
USE ROLE ACCOUNTADMIN;
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.ANALYTICS;
```

```
-- 1. Policy Generica per stringhe sensibili (es. Città)
CREATE OR REPLACE MASKING POLICY MP_PII_STRING AS (val STRING)
RETURNS STRING ->
```

```
  CASE
    WHEN val IS NULL THEN NULL
    WHEN CURRENT_ROLE() IN ('ROLE_COMPLIANCE_OFFICER', 'ROLE_DATA_ENGINEER',
'ACCOUNTADMIN') THEN val
    ELSE '*** MASKED ***'
  END;
```

```
-- 2. Policy per date di nascita (Quasi-Identifier)
-- Si usa una data fittizia standard per mantenere il tipo di dato DATE valido ed evitare
rotture nei tool di BI
```

```
CREATE OR REPLACE MASKING POLICY MP_PII_DATE AS (val DATE)
RETURNS DATE ->
```

```
  CASE
    WHEN val IS NULL THEN NULL
    WHEN CURRENT_ROLE() IN ('ROLE_COMPLIANCE_OFFICER', 'ROLE_DATA_ENGINEER',
'ACCOUNTADMIN') THEN val
    ELSE TO_DATE('1900-01-01') -- Sentinel value per utenti non privilegiati
  END;
```

```
-- Applicazione alle colonne target in DIM_PAZIENTE (Uniche colonne PII esposte in
Analytics)
```

```
ALTER TABLE DIM_PAZIENTE MODIFY COLUMN CITTA
SET MASKING POLICY MP_PII_STRING;
```

```
ALTER TABLE DIM_PAZIENTE MODIFY COLUMN DATA_NASCITA  
SET MASKING POLICY MP_PII_DATE;
```

Questo approccio è coerente con la separazione dei compiti:

- l'analista accede al layer di consumo (**ANALYTICS**) ma vede l'attributo mascherato
- il compliance officer mantiene visibilità per finalità di controllo
- il ruolo tecnico conserva la visibilità necessaria a debugging e riconciliazioni controllate

5.5 7.5 Principio del minimo privilegio e governance

L'intero impianto RBAC è costruito secondo **least privilege** e riflette lo stesso disaccoppiamento progettuale della Sezione 6:

- **separazione per layer**: ogni ruolo riceve privilegi in funzione dello scopo del layer (atterraggio, trasformazione, consumo)
- **scrittura limitata**: la scrittura è concessa ai soli ruoli tecnici, coerentemente con il fatto che PK/FK sono informational e l'integrità è garantita dalle validazioni ELT
- **consumo controllato**: l'accesso degli analisti è indirizzato verso **ANALYTICS** (star schema) e protetto da masking su attributi sensibili

In sintesi, il modello dati e il modello di sicurezza sono stati progettati congiuntamente: poiché in Snowflake i vincoli PK/FK sono **informational** e l'integrità referenziale è garantita dalla pipeline ELT (ordine di caricamento e validazioni), la scrittura manuale viene limitata ai soli ruoli tecnici. L'esposizione verso gli utenti di business avviene invece sul solo layer **ANALYTICS**, dove il dato è già normalizzato per il consumo e gli attributi sensibili della **DIM_PAZIENTE** sono protetti con masking policy.

6 Caricamento dati e pipeline

6.1 Obiettivo

In questa sezione descrivo e documento il processo operativo di caricamento e trasformazione dati su Snowflake, a partire da file **Parquet** nel Data Lake su Amazon S3. Il processo è **uno solo**, verificabile end-to-end, ed è eseguito esclusivamente tramite **TASK chain** che invoca tre procedure SQL con EXECUTE AS OWNER:

S3 → stage esterno → **RAW (COPY INTO)** → **CURATED (quarantena + MERGE)** → **ANALYTICS (MERGE)** → **orchestrazione con task chain**.

6.2 Ingestione e staging

L'ingestione avviene in modalità **pull** da S3 tramite **Storage Integration** e stage esterno, senza credenziali statiche e con **least privilege**.

A) Bucket S3 e struttura prefix

Il Data Lake risiede su Amazon S3 (regione eu-central-1) con bucket privato e **Block Public Access** attivo:

- Bucket: healthcare-data-prod-eu
- Base path: s3://healthcare-data-prod-eu/snowflake/raw/
- Prefix per domini:
 - ehr/ (es. ehr/pazienti/, ehr/ricoveri/)
 - erp/ (es. erp/reparti/)
 - iot/ (es. iot/dispositivi/, iot/parametri_vitali/)

B) IAM Role + policy (least privilege)

L'accesso cross-account da Snowflake a S3 utilizza un ruolo IAM dedicato:

- Role: snowflake_s3_readonly_role
- Policy: snowflake_s3_raw_read_policy con soli privilegi s3:ListBucket e s3:GetObject sulle risorse:
 - arn:aws:s3:::healthcare-data-prod-eu
 - arn:aws:s3:::healthcare-data-prod-eu/snowflake/raw/*

C) Storage Integration e trust policy (cross-account)

La **Storage Integration** è un oggetto di account e viene creata con privilegi amministrativi.

```
USE ROLE ACCOUNTADMIN;
```

```
CREATE OR REPLACE STORAGE INTEGRATION s3_int_healthcare_data
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = S3
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowflake_s3_readonly_role'
  STORAGE_ALLOWED_LOCATIONS = ('s3://healthcare-data-prod-eu/snowflake/raw/');
```

Codice 12: Creazione Storage Integration per accesso a S3

I parametri necessari alla trust policy AWS vengono recuperati dall'integrazione:

```
DESC INTEGRATION s3_int_healthcare_data;
```

Codice 13: Recupero parametri per la trust policy (cross-account)

Dal risultato si copiano i valori:

- STORAGE_AWS_IAM_USER_ARN
- STORAGE_AWS_EXTERNAL_ID

La trust policy del ruolo IAM utilizza questi due valori per prevenire **confused deputy**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSnowflakeAssumeRole",
      "Effect": "Allow",
      "Principal": {
        "AWS": "<SNOWFLAKE_IAM_USER_ARN>"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "<SNOWFLAKE_EXTERNAL_ID>"
        }
      }
    }
  ]
}
```

Codice 14: Trust policy del ruolo IAM (valori Snowflake reali sostituiti con placeholder)

D) File format Parquet e stage esterno

File format e stage esterno sono creati nello schema RAW del database HEALTHCARE_DW.


```

USE ROLE ACCOUNTADMIN;
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.RAW;

CREATE OR REPLACE FILE FORMAT ff_parquet
  TYPE = PARQUET;

CREATE OR REPLACE STAGE stage_healthcare_raw
  URL = 's3://healthcare-data-prod-eu/snowflake/raw/'
  STORAGE_INTEGRATION = s3_int_healthcare_data
  FILE_FORMAT = ff_parquet;

```

Codice 15: Creazione file format Parquet e stage esterno su S3

E) Verifica stage (evidenza di listing)

Le evidenze operative sono SHOW/DESC e LIST sul prefix atteso.

```

SHOW STAGES LIKE 'STAGE_HEALTHCARE_RAW' IN SCHEMA HEALTHCARE_DW.RAW;
DESC STAGE HEALTHCARE_DW.RAW.stage_healthcare_raw;
LIST @HEALTHCARE_DW.RAW.stage_healthcare_raw/ehr/pazienti/;

```

6.3 Pipeline ELT (flusso unico: task → procedure)

Il processo ELT è eseguito **solo** tramite task chain nello schema tecnico HEALTHCARE_DW.PIPELINE. Le tre procedure sono idempotenti e vengono invocate in sequenza fissa:

1. `sp_load_raw()` → carica RAW.* con COPY INTO (fail-fast).
2. `sp_transform_curated()` → quarantena e popolamento CURATED.* esclusivamente via MERGE.
3. `sp_publish_analytics()` → popolamento ANALYTICS.* esclusivamente via MERGE.

6.3.1 1) RAW: sp_load_raw() (COPY INTO, fail-fast)

La procedura di ingestione usa WH_INGEST e carica le tabelle mirror nel layer RAW. Ogni COPY INTO opera con ON_ERROR = 'ABORT_STATEMENT' per interrompere in modo deterministico su incompatibilità schema/file.

```

USE ROLE ROLE_DATA_ENGINEER;
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.PIPELINE;

CREATE OR REPLACE PROCEDURE sp_load_raw()
  RETURNS STRING
  LANGUAGE SQL
  EXECUTE AS OWNER
AS
$$
BEGIN
  COPY INTO HEALTHCARE_DW.RAW.PAZIENTI
  FROM @HEALTHCARE_DW.RAW.stage_healthcare_raw/ehr/pazienti/
  FILE_FORMAT = (TYPE = PARQUET)
  MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
  ON_ERROR = 'ABORT_STATEMENT';

  COPY INTO HEALTHCARE_DW.RAW.REPARTI
  FROM @HEALTHCARE_DW.RAW.stage_healthcare_raw/erp/reparti/
  FILE_FORMAT = (TYPE = PARQUET)
  MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
  ON_ERROR = 'ABORT_STATEMENT';

  COPY INTO HEALTHCARE_DW.RAW.DISPOSITIVI
  FROM @HEALTHCARE_DW.RAW.stage_healthcare_raw/iot/dispositivi/

```

```

FILE_FORMAT = (TYPE = PARQUET)
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
ON_ERROR = 'ABORT_STATEMENT';

COPY INTO HEALTHCARE_DW.RAW.RICOVERI
FROM @HEALTHCARE_DW.RAW.stage_healthcare_raw/ehr/ricoveri/
FILE_FORMAT = (TYPE = PARQUET)
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
ON_ERROR = 'ABORT_STATEMENT';

COPY INTO HEALTHCARE_DW.RAW.PARAMETRI_VITALI
FROM @HEALTHCARE_DW.RAW.stage_healthcare_raw/iot/parametri_vitali/
FILE_FORMAT = (TYPE = PARQUET)
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE
ON_ERROR = 'ABORT_STATEMENT';

RETURN 'OK';
END;
$$;

```

6.3.2 2) CURATED: sp_transform_curated() (quarantena + MERGE, idempotente)

Il layer CURATED viene popolato esclusivamente via MERGE sulle business key. Le anomalie (chiavi nulle e/o riferimenti orfani) vengono isolate solo tramite INSERT nelle tabelle *_QUARANTENA.

```

USE ROLE ROLE_DATA_ENGINEER;
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.PIPELINE;

CREATE OR REPLACE PROCEDURE sp_transform_curated()
RETURNS STRING
LANGUAGE SQL
EXECUTE AS OWNER
AS
$$
BEGIN
  -- a) Quarantena come snapshot per run
  TRUNCATE TABLE HEALTHCARE_DW.CURATED.RICOVERI_QUARANTENA;
  TRUNCATE TABLE HEALTHCARE_DW.CURATED.PARAMETRI_VITALI_QUARANTENA;

  -- b) MERGE CURATED.PAZIENTI (dedup deterministica su ID_PAZIENTE)
  MERGE INTO HEALTHCARE_DW.CURATED.PAZIENTI t
  USING (
    SELECT
      r.ID_PAZIENTE,
      r.CODICE_FISCALE,
      r.NOME,
      r.COGNOME,
      r.SESSO,
      r.DATA_NASCITA,
      r.STATO_CIVILE,
      r.LINGUA_PRIMARIA,
      r.GRUPPO_SANGUIGNO,
      r.CITTA,
      r.INDIRIZZO,
      r.CAP,
      r.PAESE,
      r.EMAIL,
      r.TELEFONO,
      r.COMPAGNIA_ASSICURATIVA,

```

```

    r.PIANO_ASSICURATIVO,
    r.ID_ASSICURAZIONE,
    r.CONTATTO_EMERGENZA_NOME,
    r.CONTATTO_EMERGENZA_TELEFONO,
    r.ALTEZZA_CM,
    r.PESO_KG
FROM HEALTHCARE_DW.RAW.PAZIENTI r
WHERE r.ID_PAZIENTE IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY r.ID_PAZIENTE
    ORDER BY MD5(
        TO_VARCHAR(
            OBJECT_CONSTRUCT_KEEP_NULL(
                'CODICE_FISCALE', r.CODICE_FISCALE,
                'NOME', r.NOME,
                'COGNOME', r.COGNOME,
                'SESSO', r.SESSO,
                'DATA_NASCITA', r.DATA_NASCITA,
                'GRUPPO_SANGUIGNO', r.GRUPPO_SANGUIGNO,
                'CITTA', r.CITTA,
                'INDIRIZZO', r.INDIRIZZO,
                'CAP', r.CAP,
                'PAESE', r.PAESE,
                'EMAIL', r.EMAIL,
                'TELEFONO', r.TELEFONO,
                'PIANO_ASSICURATIVO', r.PIANO_ASSICURATIVO,
                'ID_ASSICURAZIONE', r.ID_ASSICURAZIONE
            )
        )
    ) DESC
) = 1
) s
ON t.ID_PAZIENTE = s.ID_PAZIENTE
WHEN MATCHED THEN UPDATE SET
    CODICE_FISCALE = s.CODICE_FISCALE,
    NOME = s.NOME,
    COGNOME = s.COGNOME,
    SESSO = s.SESSO,
    DATA_NASCITA = s.DATA_NASCITA,
    STATO_CIVILE = s.STATO_CIVILE,
    LINGUA_PRIMARIA = s.LINGUA_PRIMARIA,
    GRUPPO_SANGUIGNO = s.GRUPPO_SANGUIGNO,
    CITTA = s.CITTA,
    INDIRIZZO = s.INDIRIZZO,
    CAP = s.CAP,
    PAESE = s.PAESE,
    EMAIL = s.EMAIL,
    TELEFONO = s.TELEFONO,
    COMPAGNIA_ASSICURATIVA = s.COMPAGNIA_ASSICURATIVA,
    PIANO_ASSICURATIVO = s.PIANO_ASSICURATIVO,
    ID_ASSICURAZIONE = s.ID_ASSICURAZIONE,
    CONTATTO_EMERGENZA_NOME = s.CONTATTO_EMERGENZA_NOME,
    CONTATTO_EMERGENZA_TELEFONO = s.CONTATTO_EMERGENZA_TELEFONO,
    ALTEZZA_CM = s.ALTEZZA_CM,
    PESO_KG = s.PESO_KG
WHEN NOT MATCHED THEN INSERT (
    ID_PAZIENTE,
    CODICE_FISCALE,
    NOME,

```

```

COGNOME,
SESSO,
DATA_NASCITA,
STATO_CIVILE,
LINGUA_PRIMARIA,
GRUPPO_SANGUIGNO,
CITTA,
INDIRIZZO,
CAP,
PAESE,
EMAIL,
TELEFONO,
COMPAGNIA_ASSICURATIVA,
PIANO_ASSICURATIVO,
ID_ASSICURAZIONE,
CONTATTO_EMERGENZA_NOME,
CONTATTO_EMERGENZA_TELEFONO,
ALTEZZA_CM,
PESO_KG
)
VALUES (
  s.ID_PAZIENTE,
  s.CODICE_FISCALE,
  s.NOME,
  s.COGNOME,
  s.SESSO,
  s.DATA_NASCITA,
  s.STATO_CIVILE,
  s.LINGUA_PRIMARIA,
  s.GRUPPO_SANGUIGNO,
  s.CITTA,
  s.INDIRIZZO,
  s.CAP,
  s.PAESE,
  s.EMAIL,
  s.TELEFONO,
  s.COMPAGNIA_ASSICURATIVA,
  s.PIANO_ASSICURATIVO,
  s.ID_ASSICURAZIONE,
  s.CONTATTO_EMERGENZA_NOME,
  s.CONTATTO_EMERGENZA_TELEFONO,
  s.ALTEZZA_CM,
  s.PESO_KG
);

-- c) MERGE CURATED.REPARTI (dedup deterministica su ID_REPARTO)
MERGE INTO HEALTHCARE_DW.CURATED.REPARTI t
USING (
  SELECT
    r.ID_REPARTO,
    r.NOME_REPARTO,
    r.SPECIALITA
  FROM HEALTHCARE_DW.RAW.REPARTI r
  WHERE r.ID_REPARTO IS NOT NULL
  QUALIFY ROW_NUMBER() OVER (
    PARTITION BY r.ID_REPARTO
    ORDER BY MD5(
      TO_VARCHAR(
        OBJECT_CONSTRUCT_KEEP_NULL(

```

```

        'NOME_REPARTO', r.NOME_REPARTO,
        'SPECIALITA', r.SPECIALITA
    )
    )
    ) DESC
    ) = 1
) s
ON t.ID_REPARTO = s.ID_REPARTO
WHEN MATCHED THEN UPDATE SET
    NOME_REPARTO = s.NOME_REPARTO,
    SPECIALITA = s.SPECIALITA
WHEN NOT MATCHED THEN INSERT (ID_REPARTO, NOME_REPARTO, SPECIALITA)
VALUES (s.ID_REPARTO, s.NOME_REPARTO, s.SPECIALITA);

-- d) Quarantena RICOVERI: PK/FK NULL e/o orfani (LEFT JOIN verso CURATED)
-- MUST-FIX: quarantena deduplicata e deterministica (business key + fallback key su PK
NULL)
INSERT INTO HEALTHCARE_DW.CURATED.RICOVERI_QUARANTENA
SELECT q.* EXCLUDE (QUARANTINE_KEY, ROW_FINGERPRINT)
FROM (
    SELECT
        r.*,
        COALESCE(
            TO_VARCHAR(r.ID_RICOVERO),
            MD5(
                TO_VARCHAR(
                    OBJECT_CONSTRUCT_KEEP_NULL(
                        'ID_RICOVERO', r.ID_RICOVERO,
                        'ID_PAZIENTE', r.ID_PAZIENTE,
                        'ID_REPARTO', r.ID_REPARTO,
                        'DATA_RICOVERO', r.DATA_RICOVERO,
                        'DATA_DIMISSIONE', r.DATA_DIMISSIONE,
                        'DURATA_DEGENZA_GIORNI', r.DURATA_DEGENZA_GIORNI,
                        'TIPO_RICOVERO', r.TIPO_RICOVERO,
                        'PROVENIENZA_RICOVERO', r.PROVENIENZA_RICOVERO,
                        'ESITO_DIMISSIONE', r.ESITO_DIMISSIONE
                    )
                )
            )
        )
    ) AS QUARANTINE_KEY,
    MD5(
        TO_VARCHAR(
            OBJECT_CONSTRUCT_KEEP_NULL(
                'ID_PAZIENTE', r.ID_PAZIENTE,
                'ID_REPARTO', r.ID_REPARTO,
                'DATA_RICOVERO', r.DATA_RICOVERO,
                'DATA_DIMISSIONE', r.DATA_DIMISSIONE,
                'DURATA_DEGENZA_GIORNI', r.DURATA_DEGENZA_GIORNI,
                'TIPO_RICOVERO', r.TIPO_RICOVERO,
                'PROVENIENZA_RICOVERO', r.PROVENIENZA_RICOVERO,
                'ESITO_DIMISSIONE', r.ESITO_DIMISSIONE
            )
        )
    ) AS ROW_FINGERPRINT
FROM HEALTHCARE_DW.RAW.RICOVERI r
LEFT JOIN HEALTHCARE_DW.CURATED.PAZIENTI p
    ON r.ID_PAZIENTE = p.ID_PAZIENTE
LEFT JOIN HEALTHCARE_DW.CURATED.REPARTI rep
    ON r.ID_REPARTO = rep.ID_REPARTO

```

```

WHERE r.ID_RICOVERO IS NULL
      OR r.ID_PAZIENTE IS NULL
      OR r.ID_REPARTO IS NULL
      OR p.ID_PAZIENTE IS NULL
      OR rep.ID_REPARTO IS NULL
) q
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY q.QUARANTINE_KEY
  -- MUST-FIX: fallback timestamp coerente, deterministico e robusto
  ORDER BY COALESCE(
    TRY_TO_TIMESTAMP_NTZ(TO_VARCHAR(q.DATA_RICOVERO)),
    TO_TIMESTAMP_NTZ('1900-01-01 00:00:00')
  ) DESC,
  q.ROW_FINGERPRINT DESC
) = 1;

-- e) MERGE CURATED.RICOVERI (join valido verso CURATED.PAZIENTI + CURATED.REPARTI)
MERGE INTO HEALTHCARE_DW.CURATED.RICOVERI t
USING (
  SELECT
    r.ID_RICOVERO,
    r.ID_PAZIENTE,
    r.ID_REPARTO,
    r.DATA_RICOVERO,
    r.DATA_DIMISSIONE,
    r.DURATA_DEGENZA_GIORNI,
    r.TIPO_RICOVERO,
    r.PROVENIENZA_RICOVERO,
    r.ESITO_DIMISSIONE
  FROM HEALTHCARE_DW.RAW.RICOVERI r
  JOIN HEALTHCARE_DW.CURATED.PAZIENTI p
    ON r.ID_PAZIENTE = p.ID_PAZIENTE
  JOIN HEALTHCARE_DW.CURATED.REPARTI rep
    ON r.ID_REPARTO = rep.ID_REPARTO
  WHERE r.ID_RICOVERO IS NOT NULL
        AND r.ID_PAZIENTE IS NOT NULL
        AND r.ID_REPARTO IS NOT NULL
  QUALIFY ROW_NUMBER() OVER (
    PARTITION BY r.ID_RICOVERO
    ORDER BY r.DATA_RICOVERO DESC,
      MD5(
        TO_VARCHAR(
          OBJECT_CONSTRUCT_KEEP_NULL(
            'ID_PAZIENTE', r.ID_PAZIENTE,
            'ID_REPARTO', r.ID_REPARTO,
            'DATA_RICOVERO', r.DATA_RICOVERO,
            'DATA_DIMISSIONE', r.DATA_DIMISSIONE,
            'DURATA_DEGENZA_GIORNI', r.DURATA_DEGENZA_GIORNI,
            'TIPO_RICOVERO', r.TIPO_RICOVERO,
            'PROVENIENZA_RICOVERO', r.PROVENIENZA_RICOVERO,
            'ESITO_DIMISSIONE', r.ESITO_DIMISSIONE
          )
        )
      ) DESC
  ) = 1
) s
ON t.ID_RICOVERO = s.ID_RICOVERO
WHEN MATCHED THEN UPDATE SET
  ID_PAZIENTE = s.ID_PAZIENTE,

```

```

ID_REPARTO = s.ID_REPARTO,
DATA_RICOVERO = s.DATA_RICOVERO,
DATA_DIMISSIONE = s.DATA_DIMISSIONE,
DURATA_DEGENZA_GIORNI = s.DURATA_DEGENZA_GIORNI,
TIPO_RICOVERO = s.TIPO_RICOVERO,
PROVENIENZA_RICOVERO = s.PROVENIENZA_RICOVERO,
ESITO_DIMISSIONE = s.ESITO_DIMISSIONE
WHEN NOT MATCHED THEN INSERT (
    ID_RICOVERO,
    ID_PAZIENTE,
    ID_REPARTO,
    DATA_RICOVERO,
    DATA_DIMISSIONE,
    DURATA_DEGENZA_GIORNI,
    TIPO_RICOVERO,
    PROVENIENZA_RICOVERO,
    ESITO_DIMISSIONE
)
VALUES (
    s.ID_RICOVERO,
    s.ID_PAZIENTE,
    s.ID_REPARTO,
    s.DATA_RICOVERO,
    s.DATA_DIMISSIONE,
    s.DURATA_DEGENZA_GIORNI,
    s.TIPO_RICOVERO,
    s.PROVENIENZA_RICOVERO,
    s.ESITO_DIMISSIONE
);

-- f) Quarantena PARAMETRI_VITALI: PK/FK NULL e/o orfani (LEFT JOIN verso
CURATED.PAZIENTI e RAW.DISPOSITIVI)
-- MUST-FIX: quarantena deduplicata e deterministica (business key + fallback key su PK
NULL)
INSERT INTO HEALTHCARE_DW.CURATED.PARAMETRI_VITALI_QUARANTENA
SELECT q.* EXCLUDE (QUARANTINE_KEY, ROW_FINGERPRINT)
FROM (
    SELECT
        v.*,
        COALESCE(
            TO_VARCHAR(v.ID_MISURAZIONE),
            MD5(
                TO_VARCHAR(
                    OBJECT_CONSTRUCT_KEEP_NULL(
                        'ID_MISURAZIONE', v.ID_MISURAZIONE,
                        'ID_PAZIENTE', v.ID_PAZIENTE,
                        'ID_DISPOSITIVO', v.ID_DISPOSITIVO,
                        'DATA_MISURAZIONE', v.DATA_MISURAZIONE,
                        'FREQUENZA_CARDIACA', v.FREQUENZA_CARDIACA,
                        'SATURAZIONE_OSSIGENO', v.SATURAZIONE_OSSIGENO,
                        'PRESSIONE_SISTOLICA', v.PRESSIONE_SISTOLICA,
                        'PRESSIONE_DIASTOLICA', v.PRESSIONE_DIASTOLICA,
                        'TEMPERATURA_C', v.TEMPERATURA_C,
                        'FREQUENZA_RESPIRATORIA', v.FREQUENZA_RESPIRATORIA,
                        'GLICEMIA_MG_DL', v.GLICEMIA_MG_DL
                    )
                )
            )
        ) AS QUARANTINE_KEY,

```

```

MD5(
  TO_VARCHAR(
    OBJECT_CONSTRUCT_KEEP_NULL(
      'ID_PAZIENTE', v.ID_PAZIENTE,
      'ID_DISPOSITIVO', v.ID_DISPOSITIVO,
      'DATA_MISURAZIONE', v.DATA_MISURAZIONE,
      'FREQUENZA_CARDIACA', v.FREQUENZA_CARDIACA,
      'SATURAZIONE_OSSIGENO', v.SATURAZIONE_OSSIGENO,
      'PRESSIONE_SISTOLICA', v.PRESSIONE_SISTOLICA,
      'PRESSIONE_DIASTOLICA', v.PRESSIONE_DIASTOLICA,
      'TEMPERATURA_C', v.TEMPERATURA_C,
      'FREQUENZA_RESPIRATORIA', v.FREQUENZA_RESPIRATORIA,
      'GLICEMIA_MG_DL', v.GLICEMIA_MG_DL
    )
  )
) AS ROW_FINGERPRINT
FROM HEALTHCARE_DW.RAW.PARAMETRI_VITALI v
LEFT JOIN HEALTHCARE_DW.CURATED.PAZIENTI p
  ON v.ID_PAZIENTE = p.ID_PAZIENTE
LEFT JOIN HEALTHCARE_DW.RAW.DISPOSITIVI d
  ON v.ID_DISPOSITIVO = d.ID_DISPOSITIVO
WHERE v.ID_MISURAZIONE IS NULL
      OR v.ID_PAZIENTE IS NULL
      OR v.ID_DISPOSITIVO IS NULL
      OR p.ID_PAZIENTE IS NULL
      OR d.ID_DISPOSITIVO IS NULL
) q
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY q.QUARANTINE_KEY
  -- MUST-FIX: fallback timestamp coerente, deterministico e robusto
  ORDER BY COALESCE(
    TRY_TO_TIMESTAMP_NTZ(TO_VARCHAR(q.DATA_MISURAZIONE)),
    TO_TIMESTAMP_NTZ('1900-01-01 00:00:00')
  ) DESC,
  q.ROW_FINGERPRINT DESC
) = 1;

-- g) MERGE CURATED.PARAMETRI_VITALI (join valido verso CURATED.PAZIENTI +
RAW.DISPOSITIVI)
MERGE INTO HEALTHCARE_DW.CURATED.PARAMETRI_VITALI t
USING (
  SELECT
    v.ID_MISURAZIONE,
    v.ID_PAZIENTE,
    v.ID_DISPOSITIVO,
    v.DATA_MISURAZIONE,
    v.FREQUENZA_CARDIACA,
    v.SATURAZIONE_OSSIGENO,
    v.PRESSIONE_SISTOLICA,
    v.PRESSIONE_DIASTOLICA,
    v.TEMPERATURA_C,
    v.FREQUENZA_RESPIRATORIA,
    v.GLICEMIA_MG_DL
  FROM HEALTHCARE_DW.RAW.PARAMETRI_VITALI v
  JOIN HEALTHCARE_DW.CURATED.PAZIENTI p
    ON v.ID_PAZIENTE = p.ID_PAZIENTE
  JOIN HEALTHCARE_DW.RAW.DISPOSITIVI d
    ON v.ID_DISPOSITIVO = d.ID_DISPOSITIVO
  WHERE v.ID_MISURAZIONE IS NOT NULL

```



```

    AND v.ID_PAZIENTE IS NOT NULL
    AND v.ID_DISPOSITIVO IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY v.ID_MISURAZIONE
    ORDER BY v.DATA_MISURAZIONE DESC,
    MD5(
        TO_VARCHAR(
            OBJECT_CONSTRUCT_KEEP_NULL(
                'ID_PAZIENTE', v.ID_PAZIENTE,
                'ID_DISPOSITIVO', v.ID_DISPOSITIVO,
                'DATA_MISURAZIONE', v.DATA_MISURAZIONE,
                'FREQUENZA_CARDIACA', v.FREQUENZA_CARDIACA,
                'SATURAZIONE_OSSIGENO', v.SATURAZIONE_OSSIGENO,
                'PRESSIONE_SISTOLICA', v.PRESSIONE_SISTOLICA,
                'PRESSIONE_DIASTOLICA', v.PRESSIONE_DIASTOLICA,
                'TEMPERATURA_C', v.TEMPERATURA_C,
                'FREQUENZA_RESPIRATORIA', v.FREQUENZA_RESPIRATORIA,
                'GLICEMIA_MG_DL', v.GLICEMIA_MG_DL
            )
        )
    ) DESC
) = 1
) s
ON t.ID_MISURAZIONE = s.ID_MISURAZIONE
WHEN MATCHED THEN UPDATE SET
    ID_PAZIENTE = s.ID_PAZIENTE,
    ID_DISPOSITIVO = s.ID_DISPOSITIVO,
    DATA_MISURAZIONE = s.DATA_MISURAZIONE,
    FREQUENZA_CARDIACA = s.FREQUENZA_CARDIACA,
    SATURAZIONE_OSSIGENO = s.SATURAZIONE_OSSIGENO,
    PRESSIONE_SISTOLICA = s.PRESSIONE_SISTOLICA,
    PRESSIONE_DIASTOLICA = s.PRESSIONE_DIASTOLICA,
    TEMPERATURA_C = s.TEMPERATURA_C,
    FREQUENZA_RESPIRATORIA = s.FREQUENZA_RESPIRATORIA,
    GLICEMIA_MG_DL = s.GLICEMIA_MG_DL
WHEN NOT MATCHED THEN INSERT (
    ID_MISURAZIONE,
    ID_PAZIENTE,
    ID_DISPOSITIVO,
    DATA_MISURAZIONE,
    FREQUENZA_CARDIACA,
    SATURAZIONE_OSSIGENO,
    PRESSIONE_SISTOLICA,
    PRESSIONE_DIASTOLICA,
    TEMPERATURA_C,
    FREQUENZA_RESPIRATORIA,
    GLICEMIA_MG_DL
)
VALUES (
    s.ID_MISURAZIONE,
    s.ID_PAZIENTE,
    s.ID_DISPOSITIVO,
    s.DATA_MISURAZIONE,
    s.FREQUENZA_CARDIACA,
    s.SATURAZIONE_OSSIGENO,
    s.PRESSIONE_SISTOLICA,
    s.PRESSIONE_DIASTOLICA,
    s.TEMPERATURA_C,
    s.FREQUENZA_RESPIRATORIA,

```

```

        s.GLICEMIA_MG_DL
    );

    RETURN 'OK';
END;
$$;

```

6.3.3 3) ANALYTICS: sp__publish_analytics() (solo MERGE, idempotente)

La procedura esegue esclusivamente MERGE idempotenti sulle business key, senza DDL e senza INSERT di popolamento.

```

USE ROLE ROLE_DATA_ENGINEER;
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.PIPELINE;

```

```

CREATE OR REPLACE PROCEDURE sp_publish_analytics()
    RETURNS STRING
    LANGUAGE SQL
    EXECUTE AS OWNER
AS
$$
BEGIN
    -- a) DIM
    MERGE INTO HEALTHCARE_DW.ANALYTICS.DIM_PAZIENTE t
    USING (
        SELECT
            ID_PAZIENTE,
            SESSO,
            DATA_NASCITA,
            CITTA,
            GRUPPO_SANGUIGNO,
            PIANO_ASSICURATIVO
        FROM HEALTHCARE_DW.CURATED.PAZIENTI
    ) s
    ON t.ID_PAZIENTE = s.ID_PAZIENTE
    WHEN MATCHED THEN UPDATE SET
        SESSO = s.SESSO,
        DATA_NASCITA = s.DATA_NASCITA,
        CITTA = s.CITTA,
        GRUPPO_SANGUIGNO = s.GRUPPO_SANGUIGNO,
        PIANO_ASSICURATIVO = s.PIANO_ASSICURATIVO
    WHEN NOT MATCHED THEN INSERT (ID_PAZIENTE, SESSO, DATA_NASCITA, CITTA, GRUPPO_SANGUIGNO,
    PIANO_ASSICURATIVO)
    VALUES (s.ID_PAZIENTE, s.SESSO, s.DATA_NASCITA, s.CITTA, s.GRUPPO_SANGUIGNO,
    s.PIANO_ASSICURATIVO);

    MERGE INTO HEALTHCARE_DW.ANALYTICS.DIM_REPARTO t
    USING (
        SELECT
            ID_REPARTO,
            NOME_REPARTO,
            SPECIALITA
        FROM HEALTHCARE_DW.CURATED.REPARTI
    ) s
    ON t.ID_REPARTO = s.ID_REPARTO
    WHEN MATCHED THEN UPDATE SET
        NOME_REPARTO = s.NOME_REPARTO,
        SPECIALITA = s.SPECIALITA
    WHEN NOT MATCHED THEN INSERT (ID_REPARTO, NOME_REPARTO, SPECIALITA)

```

```

VALUES (s.ID_REPARTO, s.NOME_REPARTO, s.SPECIALITA);

-- b) FACT_RICOVERI
MERGE INTO HEALTHCARE_DW.ANALYTICS.FACT_RICOVERI t
USING (
    SELECT
        ID_RICOVERO,
        ID_PAZIENTE,
        ID_REPARTO,
        DATA_RICOVERO AS TS_RICOVERO,
        DATA_DIMISSIONE AS TS_DIMISSIONE,
        DURATA_DEGENZA_GIORNI,
        TIPO_RICOVERO,
        ESITO_DIMISSIONE
    FROM HEALTHCARE_DW.CURATED.RICOVERI
) s
ON t.ID_RICOVERO = s.ID_RICOVERO
WHEN MATCHED THEN UPDATE SET
    ID_PAZIENTE = s.ID_PAZIENTE,
    ID_REPARTO = s.ID_REPARTO,
    TS_RICOVERO = s.TS_RICOVERO,
    TS_DIMISSIONE = s.TS_DIMISSIONE,
    DURATA_DEGENZA_GIORNI = s.DURATA_DEGENZA_GIORNI,
    TIPO_RICOVERO = s.TIPO_RICOVERO,
    ESITO_DIMISSIONE = s.ESITO_DIMISSIONE
WHEN NOT MATCHED THEN INSERT (
    ID_RICOVERO,
    ID_PAZIENTE,
    ID_REPARTO,
    TS_RICOVERO,
    TS_DIMISSIONE,
    DURATA_DEGENZA_GIORNI,
    TIPO_RICOVERO,
    ESITO_DIMISSIONE
)
VALUES (
    s.ID_RICOVERO,
    s.ID_PAZIENTE,
    s.ID_REPARTO,
    s.TS_RICOVERO,
    s.TS_DIMISSIONE,
    s.DURATA_DEGENZA_GIORNI,
    s.TIPO_RICOVERO,
    s.ESITO_DIMISSIONE
);

-- c) FACT_MISURAZIONI (ORA_GIORNO derivata)
MERGE INTO HEALTHCARE_DW.ANALYTICS.FACT_MISURAZIONI t
USING (
    SELECT
        ID_MISURAZIONE,
        ID_PAZIENTE,
        ID_DISPOSITIVO,
        DATA_MISURAZIONE AS TS_MISURAZIONE,
        EXTRACT(HOUR FROM DATA_MISURAZIONE) AS ORA_GIORNO,
        FREQUENZA_CARDIACA,
        SATURAZIONE_OSSIGENO,
        PRESSIONE_SISTOLICA,
        PRESSIONE_DIASTOLICA,

```

```

        TEMPERATURA_C,
        GLICEMIA_MG_DL
    FROM HEALTHCARE_DW.CURATED.PARAMETRI_VITALI
) s
ON t.ID_MISURAZIONE = s.ID_MISURAZIONE
WHEN MATCHED THEN UPDATE SET
    ID_PAZIENTE = s.ID_PAZIENTE,
    ID_DISPOSITIVO = s.ID_DISPOSITIVO,
    TS_MISURAZIONE = s.TS_MISURAZIONE,
    ORA_GIORNO = s.ORA_GIORNO,
    FREQUENZA_CARDIACA = s.FREQUENZA_CARDIACA,
    SATURAZIONE_OSSIGENO = s.SATURAZIONE_OSSIGENO,
    PRESSIONE_SISTOLICA = s.PRESSIONE_SISTOLICA,
    PRESSIONE_DIASTOLICA = s.PRESSIONE_DIASTOLICA,
    TEMPERATURA_C = s.TEMPERATURA_C,
    GLICEMIA_MG_DL = s.GLICEMIA_MG_DL
WHEN NOT MATCHED THEN INSERT (
    ID_MISURAZIONE,
    ID_PAZIENTE,
    ID_DISPOSITIVO,
    TS_MISURAZIONE,
    ORA_GIORNO,
    FREQUENZA_CARDIACA,
    SATURAZIONE_OSSIGENO,
    PRESSIONE_SISTOLICA,
    PRESSIONE_DIASTOLICA,
    TEMPERATURA_C,
    GLICEMIA_MG_DL
)
VALUES (
    s.ID_MISURAZIONE,
    s.ID_PAZIENTE,
    s.ID_DISPOSITIVO,
    s.TS_MISURAZIONE,
    s.ORA_GIORNO,
    s.FREQUENZA_CARDIACA,
    s.SATURAZIONE_OSSIGENO,
    s.PRESSIONE_SISTOLICA,
    s.PRESSIONE_DIASTOLICA,
    s.TEMPERATURA_C,
    s.GLICEMIA_MG_DL
);

RETURN 'OK';
END;
$$;

```

6.4 Orchestrazione e scheduling (HEALTHCARE_DW.PIPELINE)

L'orchestrazione è implementata con **TASK** in catena (AFTER) nello schema tecnico HEALTHCARE_DW.PIPELINE, con scheduling giornaliero alle 02:00 (Europe/Rome) sul task root.

6.4.1 Grant strutturali (staging + schema)

```
USE ROLE SECURITYADMIN;
```

```
-- Staging (RAW)
```

```
GRANT USAGE ON FILE FORMAT HEALTHCARE_DW.RAW.ff_parquet TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON STAGE HEALTHCARE_DW.RAW.stage_healthcare_raw TO ROLE ROLE_DATA_ENGINEER;
```

Task chain: procedure → task root schedulato → task downstream

Le procedure sono richiamate dai task e sono definite con EXECUTE AS OWNER. La sequenza di creazione è: procedure → task chain → grant runtime.

```
USE ROLE ROLE_DATA_ENGINEER;
USE DATABASE HEALTHCARE_DW;
USE SCHEMA HEALTHCARE_DW.Pipeline;

CREATE OR REPLACE TASK task_load_raw
  WAREHOUSE = WH_INGEST
  SCHEDULE = 'USING CRON 0 2 * * * Europe/Rome'
AS
  CALL HEALTHCARE_DW.Pipeline.sp_load_raw();

CREATE OR REPLACE TASK task_transform_curated
  WAREHOUSE = WH_OPERATIONS
  AFTER task_load_raw
AS
  CALL HEALTHCARE_DW.Pipeline.sp_transform_curated();

CREATE OR REPLACE TASK task_publish_analytics
  WAREHOUSE = WH_ANALYTICS
  AFTER task_transform_curated
AS
  CALL HEALTHCARE_DW.Pipeline.sp_publish_analytics();
```

6.4.2 Grant runtime (warehouse + execute + operate)

Questi grant rendono eseguibile la pipeline con ROLE_DATA_ENGINEER (warehouse, database/schema e controllo operativo dei task tramite OPERATE). Il privilegio OPERATE governa l'operatività (resume/suspend/execute/monitor) senza concedere privilegi amministrativi più ampi.

```
USE ROLE SECURITYADMIN;

-- Warehouse
GRANT USAGE ON WAREHOUSE WH_INGEST TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON WAREHOUSE WH_OPERATIONS TO ROLE ROLE_DATA_ENGINEER;
GRANT USAGE ON WAREHOUSE WH_ANALYTICS TO ROLE ROLE_DATA_ENGINEER;

-- Stored Procedures (USAGE, non EXECUTE)
GRANT USAGE ON PROCEDURE HEALTHCARE_DW.Pipeline.sp_load_raw()
  TO ROLE ROLE_DATA_ENGINEER;

GRANT USAGE ON PROCEDURE HEALTHCARE_DW.Pipeline.sp_transform_curated()
  TO ROLE ROLE_DATA_ENGINEER;

GRANT USAGE ON PROCEDURE HEALTHCARE_DW.Pipeline.sp_publish_analytics()
  TO ROLE ROLE_DATA_ENGINEER;

-- Task
GRANT OPERATE ON TASK HEALTHCARE_DW.Pipeline.task_load_raw
  TO ROLE ROLE_DATA_ENGINEER;

GRANT OPERATE ON TASK HEALTHCARE_DW.Pipeline.task_transform_curated
  TO ROLE ROLE_DATA_ENGINEER;

GRANT OPERATE ON TASK HEALTHCARE_DW.Pipeline.task_publish_analytics
  TO ROLE ROLE_DATA_ENGINEER;
```

```
GRANT EXECUTE TASK ON ACCOUNT TO ROLE ROLE_DATA_ENGINEER;
```

6.4.3 Ownership e delega operativa (EXECUTE AS OWNER)

- Le procedure e i task vengono creati con `ROLE_DATA_ENGINEER` (owner degli oggetti in `HEALTHCARE_DW.Pipeline`).
- `EXECUTE AS OWNER` implica che l'esecuzione delle procedure usa i privilegi dell'owner della procedura.
- Il controllo operativo della catena (`resume/suspend/execute/monitor`) viene delegato esplicitamente via `GRANT OPERATE` sui task.

Attivazione della catena e run end-to-end controllato (evidenza)

```
USE ROLE ROLE_DATA_ENGINEER;  
ALTER TASK HEALTHCARE_DW.Pipeline.task_load_raw SUSPEND;  
ALTER TASK HEALTHCARE_DW.Pipeline.task_transform_curated RESUME;  
ALTER TASK HEALTHCARE_DW.Pipeline.task_publish_analytics RESUME;
```

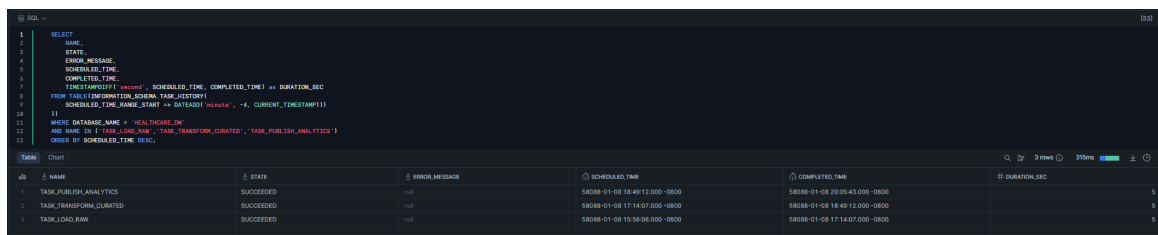
```
EXECUTE TASK HEALTHCARE_DW.Pipeline.task_load_raw;
```

Verifica esecuzione

Per confermare il corretto funzionamento della pipeline, interrogo lo storico dei task per verificare che tutti i task della catena siano in stato `SUCCEEDED`.

```
SELECT  
    NAME,  
    STATE,  
    ERROR_MESSAGE,  
    SCHEDULED_TIME,  
    COMPLETED_TIME,  
    TIMESTAMPDIFF('second', SCHEDULED_TIME, COMPLETED_TIME) as DURATION_SEC  
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(  
    SCHEDULED_TIME_RANGE_START => DATEADD('minute', -4, CURRENT_TIMESTAMP())  
))  
WHERE DATABASE_NAME = 'HEALTHCARE_DW'  
AND NAME IN ('TASK_LOAD_RAW', 'TASK_TRANSFORM_CURATED', 'TASK_PUBLISH_ANALYTICS')  
ORDER BY SCHEDULED_TIME DESC;
```

Codice 16: Query di verifica: stato di esecuzione della catena di task



id	NAME	STATE	ERROR_MESSAGE	SCHEDULED_TIME	COMPLETED_TIME	DURATION_SEC
1	TASK_PUBLISH_ANALYTICS	SUCCEEDED		2023-01-08 16:49:12.000 -0800	2023-01-08 16:49:12.000 -0800	5
2	TASK_TRANSFORM_CURATED	SUCCEEDED		2023-01-08 17:14:07.000 -0800	2023-01-08 16:49:12.000 -0800	5
3	TASK_LOAD_RAW	SUCCEEDED		2023-01-08 15:58:08.000 -0800	2023-01-08 17:14:07.000 -0800	5

Figura 2: Screenshot: evidenza dell'esecuzione a cascata (Load -> Transform -> Publish) completata con successo

7 Validazione analitica e query dimostrative

7.1 Obiettivo della validazione

In questa fase finale, eseguo una serie di interrogazioni mirate per confermare tre aspetti fondamentali del progetto:

- Completezza:** i dati sono fluiti correttamente dal livello RAW al livello ANALYTICS.
- Qualità:** la logica di quarantena ha intercettato eventuali anomalie referenziali.

3. **Sicurezza:** le policy di mascheramento dinamico proteggono i dati sensibili in base al ruolo utente.

I test vengono eseguiti impersonando i ruoli definiti nel modello di sicurezza (ROLE_DATA_ENGINEER, ROLE_DATA_ANALYST, ROLE_COMPLIANCE_OFFICER).

7.2 1. Verifica tecnica del flusso (Data Engineering)

Utilizzando il ruolo tecnico, verifico la cardinalità delle tabelle lungo la pipeline per accertarmi che non ci siano perdite di dati non giustificate (es. record scartati).

Role: ROLE_DATA_ENGINEER

```
USE ROLE ROLE_DATA_ENGINEER;  
USE WAREHOUSE WH_OPERATIONS;
```

```
-- 1. Confronto conteggi tra layer (EHR Pazienti)  
SELECT 'RAW' as LAYER, COUNT(*) as CNT FROM HEALTHCARE_DW.RAW.PAZIENTI  
UNION ALL  
SELECT 'CURATED' as LAYER, COUNT(*) as CNT FROM HEALTHCARE_DW.CURATED.PAZIENTI  
UNION ALL  
SELECT 'ANALYTICS' as LAYER, COUNT(*) as CNT FROM HEALTHCARE_DW.ANALYTICS.DIM_PAZIENTE;
```

Codice 17: Query di riconciliazione volumi tra i layer

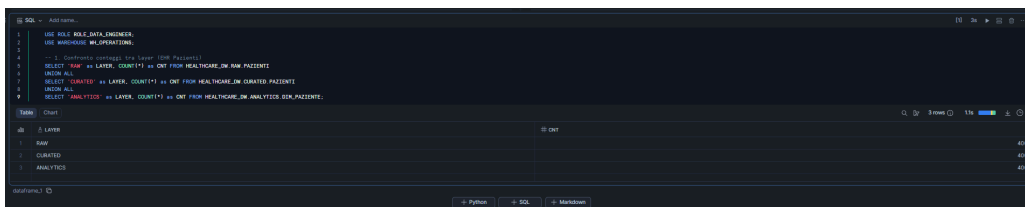


Figura 3: Screenshot: output verifica volumi (nessuna perdita di dati)

7.3 2. Verifica gestione anomalie (Quarantena)

Controllo se le tabelle di quarantena hanno catturato record che violano l'integrità referenziale (es. misurazioni di un dispositivo sconosciuto). Un risultato vuoto indica che i dati sintetici sono consistenti, ma la presenza delle tabelle garantisce robustezza.

Role: ROLE_DATA_ENGINEER

```
USE ROLE ROLE_DATA_ENGINEER;  
  
-- Verifica contenuti in quarantena  
SELECT 'RICOVERI_BAD' as TIPO, COUNT(*) as RIGHE_SCARTATE  
FROM HEALTHCARE_DW.CURATED.RICOVERI_QUARANTENA  
UNION ALL  
SELECT 'VITALS_BAD' as TIPO, COUNT(*) as RIGHE_SCARTATE  
FROM HEALTHCARE_DW.CURATED.PARAMETRI_VITALI_QUARANTENA;
```

Codice 18: Monitoraggio della Data Quality (Quarantena)

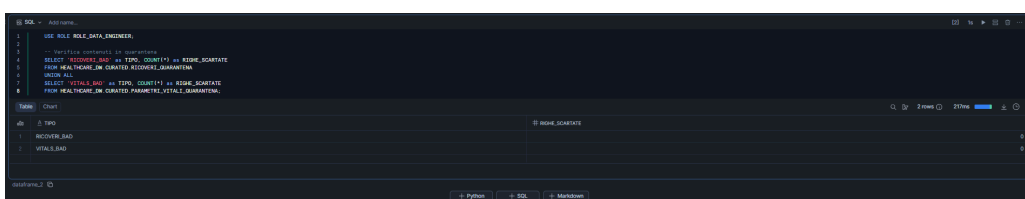


Figura 4: Screenshot: output verifica quarantena (nessuna anomalia non gestita)

7.4 3. Analisi di Business (Data Analyst)

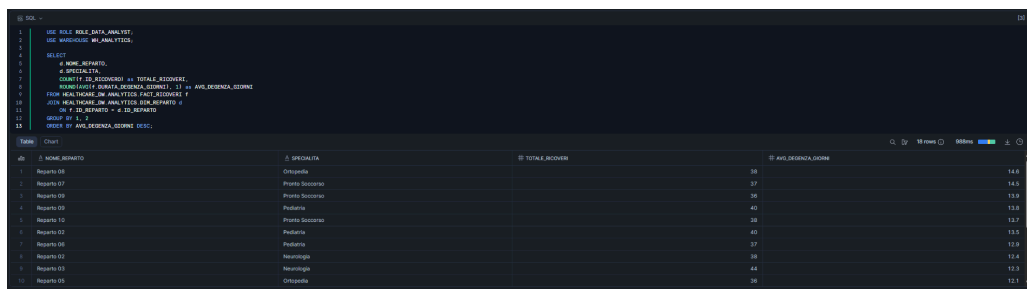
Simulo l'attività di un analista che interroga il Data Mart per estrarre insight. Questa query coinvolge Fact Table e Dimensioni, validando le Join dello Star Schema. **Usecase:** Analisi della durata media dei ricoveri (LOS) per reparto.

Role: **ROLE_DATA_ANALYST**

```
USE ROLE ROLE_DATA_ANALYST;
USE WAREHOUSE WH_ANALYTICS;

SELECT
    d.NOME_REPARTO,
    d.SPECIALITA,
    COUNT(f.ID_RICOVERO) as TOTALE_RICOVERI,
    ROUND(AVG(f.DURATA_DEGENZA_GIORNI), 1) as AVG_DEGENZA_GIORNI
FROM HEALTHCARE_DW.ANALYTICS.FACT_RICOVERI f
JOIN HEALTHCARE_DW.ANALYTICS.DIM_REPARTO d
    ON f.ID_REPARTO = d.ID_REPARTO
GROUP BY 1, 2
ORDER BY AVG_DEGENZA_GIORNI DESC;
```

Codice 19: Analisi dimensionale: Ricoveri per Reparto



Reparto	Specialità	TOTALE_RICOVERI	AVG_DEGENZA_GIORNI
Reparto 06	Ortopedia	38	14.8
Reparto 07	Pronto Soccorso	37	14.5
Reparto 09	Pronto Soccorso	36	13.9
Reparto 08	Neurologia	43	13.8
Reparto 10	Pronto Soccorso	38	13.7
Reparto 02	Pronto Soccorso	40	13.5
Reparto 03	Neurologia	37	12.8
Reparto 05	Neurologia	38	12.4
Reparto 04	Neurologia	44	12.3
Reparto 01	Ortopedia	38	12.1

Figura 5: Screenshot: risultati analisi degenza media per reparto

7.5 4. Verifica Masking Policy (Security & Compliance)

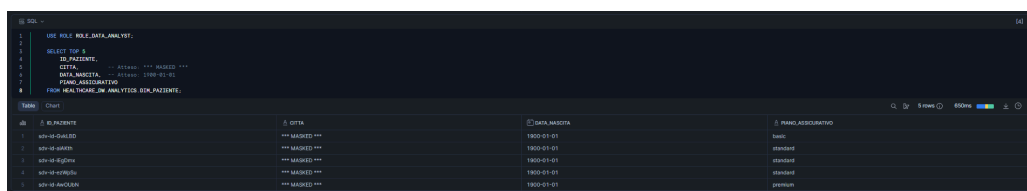
Infine, dimostro l'efficacia delle Masking Policy sui dati PII (DIM_PAZIENTE.CITTA e DATA_NASCITA).

Test 1: Vista Analista (Dati Mascherati) L'analista deve vedere **** MASKED **** e la data sentinel 1900-01-01.

```
USE ROLE ROLE_DATA_ANALYST;

SELECT TOP 5
    ID_PAZIENTE,
    CITTA, -- Atteso: ** MASKED **
    DATA_NASCITA, -- Atteso: 1900-01-01
    PIANO_ASSICURATIVO
FROM HEALTHCARE_DW.ANALYTICS.DIM_PAZIENTE;
```

Codice 20: Accesso mascherato per Data Analyst



ID_PAZIENTE	CITTA	DATA_NASCITA	PIANO_ASSICURATIVO
100-00-00000	** MASKED **	1900-01-01	basic
100-00-00001	** MASKED **	1900-01-01	basic
100-00-00002	** MASKED **	1900-01-01	basic
100-00-00003	** MASKED **	1900-01-01	basic
100-00-00004	** MASKED **	1900-01-01	premium

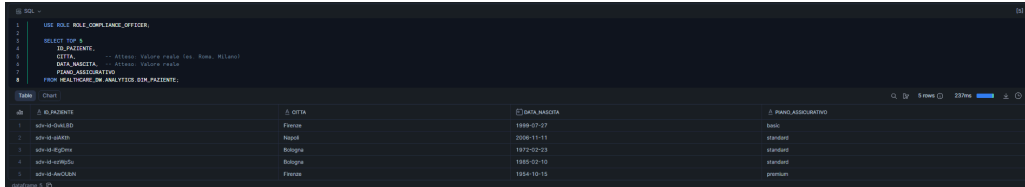
Figura 6: Screenshot: vista mascherata per ruolo Data Analyst

Test 2: Vista Compliance (Dati in Chiaro) Il Compliance Officer, avendo privilegi di audit, vede i dati originali.


```
USE ROLE ROLE_COMPLIANCE_OFFICER;
```

```
SELECT TOP 5
  ID_PAZIENTE,
  CITTA,          -- Atteso: Valore reale (es. Roma, Milano)
  DATA_NASCITA,  -- Atteso: Valore reale
  PIANO_ASSICURATIVO
FROM HEALTHCARE_DW.ANALYTICS.DIM_PAZIENTE;
```

Codice 21: Accesso unmasked per Compliance Officer



The screenshot shows a Snowflake SQL editor with a query and its results. The query is the same as the one in the previous block. The results are displayed in a table with 5 columns: ID_PAZIENTE, CITTA, DATA_NASCITA, and PIANO_ASSICURATIVO. The table contains 5 rows of data.

ID_PAZIENTE	CITTA	DATA_NASCITA	PIANO_ASSICURATIVO
1	Roma	1989-07-07	base
2	Milano	2000-11-11	standard
3	Roma	1972-02-13	standard
4	Roma	1985-02-10	standard
5	Roma	1984-10-15	premium

Figura 7: Screenshot: vista in chiaro per ruolo Compliance Officer

8 Conclusioni

8.1 Sintesi del lavoro svolto

In questa sezione conclusiva riassumo il percorso progettuale che ho portato a termine. Partendo dalla generazione di un dataset sintetico verosimile, ho attraversato le fasi di analisi dei requisiti e design architetturale, fino ad arrivare alla definizione dettagliata del modello dati su Snowflake e all'implementazione delle politiche di sicurezza.

Ogni step è stato guidato dall'obiettivo di modernizzare la gestione dei dati clinici, rispondendo alle esigenze di consolidamento e analisi espresse inizialmente.

8.2 Benefici della soluzione

La soluzione che ho progettato offre diversi vantaggi tangibili:

- **Centralizzazione:** Un'unica fonte di verità per i dati clinici elimina i silos informativi.
- **Scalabilità:** L'architettura su Snowflake garantisce prestazioni elevate anche al crescere del volume dati.
- **Sicurezza:** L'approccio Security-by-Design assicura la protezione dei dati sensibili e la conformità normativa.

8.3 Considerazioni finali

Questo progetto dimostra come una corretta ingegnerizzazione dei dati possa abilitare un miglioramento concreto nei processi sanitari. Il modello realizzato costituisce una base solida su cui poter costruire futuri sviluppi, come dashboard avanzate o modelli di machine learning predittivo, confermando il valore strategico di un'infrastruttura dati moderna e ben governata.